AI in Software Engineering

Case 03: Food Delivery MVP

Case Study Assignment: Part 1 case tasks, Part 2 theory, Part 3 labs, Appendices

Student name	[your full name]
Roll number	[your roll number]
Class / section	[your class and section]
Submission date	[date]
File name	Case03_[RollNumber]_[Name]
Process model	Prototype
Agile method	Kanban
AI tool(s)	The case names ChatGPT. The AI assistant actually used for this work was Claude (Anthropic); see Appendix A and B.
Prototype module	Cart and order placement
GitHub repository	https://github.com/GaduginaNandini/FoodDeliveryMVP (folder `case03/` holds the lab code, tests and evidence from this report)

Scenario. A start-up has 4 weeks and 3 developers to show investors ordering from 10 restaurants. **Targets used everywhere:** 4 weeks (20 working days), 3 developers, 10 restaurants, WIP limit 3, order in 4 taps (NFR-01), page load under 2 seconds (NFR-02), 12 Kanban cards, 5 test users, 10 stories, 10 manual test cases.

How to read this report

Every part uses the same IDs, so an answer in one place can be followed into another:

Prefix	Meaning	Defined in
FR-01 to FR-10	Functional requirements	Table 1 (L1.1)
NFR-01 to NFR-03	Non-functional requirements	Table 1 (L1.1)
US-01 to US-10	User stories (same order as FR)	L2.3
TC-01 to TC-10	Manual test cases	L4.1
AI-TC-01 to AI-TC-10	AI-generated test cases	L1.5
DEF-01 to DEF-07	Defects found	L4.1 and L5.3
EC-Q / EC-C	Equivalence-class tests (quantity, coupon)	Task D

Scope of the practical work

The tests, scripts and logs in this report were run on the submitted code (Django 6.1, Selenium with Chromium 153, Docker Compose with PostgreSQL, git). Where an equivalent tool or a persona was used instead (Jenkins stages run locally, pylint/radon/bandit for SonarQube), an **Evidence note** says so.

Part 1: Case Tasks

Task A (Module 1): Lifecycle and AI

Plan: two prototype iterations

	Iteration 1: clickable screens	Iteration 2: working order flow
Working days	Days 1 to 8 (weeks 1 and 2)	Days 9 to 20 (weeks 3 and 4)
Goal	Investors and test users can click through list, menu, cart, checkout and history and judge the look and feel	A real user can sign in, fill a cart, place an order and see it saved, using real data
What is built	Static Django templates with hard-coded sample data, one shared stylesheet, no database logic	Models (Restaurant, FoodItem, Order, OrderItem), seed data for 10 restaurants, session cart, login-protected checkout, order history
Stories covered	US-01, US-02, US-03 (visual only), US-05 (visual only)	US-03 to US-10 with working behaviour
Exit check	5 test users complete the “order a burger” task and answer the 10 questions below	All 10 manual test cases (TC-01 to TC-10) executed; no Critical defect open
Output	Clickable prototype and a list of changes requested by users	Working MVP demo for investors

Ten questions for 5 test users. I asked the AI assistant for questions, then removed two that led the answer (“Don’t you think the app looks modern?”) and replaced them.

1. Without help, how would you start ordering a meal here? Show me.
2. What did you expect to happen when you pressed “+ Add”? Did it happen?
3. How did you know that the item was in your cart?
4. How many taps did it take you to reach the checkout page? Did that feel fast or slow?
5. Was anything on the cart page confusing (quantity buttons, totals, price)?
6. Where would you expect to type a discount code?
7. What would make you trust the checkout page enough to enter your address and phone number?
8. Which screen felt the most polished? Which felt unfinished?
9. What would stop you from using this app instead of the one you use now?
10. If you could change one thing before we show this to investors, what would it be?

Method used in this report: persona walkthrough. I could not run a session with real users for this report, so I did not invent any. Instead I walked through the 10 questions as five **fictional personas** (not real people; the names are made up for the exercise). Each persona’s “answer” is what that kind of user would meet in the running app, based on the defects and screenshots I measured (DEF-01 to DEF-04, UX-01). This is a design technique for the early stage; it does not replace a session with real users, and the exit check for iteration 1 still requires one.

Persona (fictional)	Profile	Task given	What she meets in the app	Action
Ananya	20, student, orders lunch on her phone, low patience	Order a burger (Q1, Q4)	Finds list and menu without help; reaches checkout in 4 taps	Keep layout (FR-01, FR-02); keep NFR-01
Priya	27, office worker, orders for 3 colleagues	Add 3 items (Q2, Q3)	Nothing on screen changes after “+ Add”; unsure the item was added (UX-01)	Add a cart count and a short confirmation (new card)
Kavya	35, careful with money, always uses coupons	Find a discount box (Q6)	Looks for a coupon box before checkout and finds none	Build FR-06 (US-06)

Persona (fictional)	Profile	Task given	What she meets in the app	Action
Meera	42, first time using a delivery app, worried about typing her address	Check out (Q7)	Sees a blank total before entering address and phone (DEF-03)	Fix checkout total (DEF-03)
Divya	24, tester-minded, tries the app as a guest and then signs in	Open the app without signing in (Q8, Q10)	Home page crashes for a visitor (DEF-01); sign-in page errors (DEF-02)	Fix DEF-01 to DEF-02 first

Reading the result. Three of the five personas hit a problem in the very first minute (Priya, Meera, Divya), so the first iteration's exit check cannot pass until DEF-01 to DEF-03 are fixed. Both the persona method and its findings come from the code I tested, so a real-user session may find other problems (for example cart image or the look of the dark theme). The questions above are ready to use for that session.

What code is thrown away and what is kept.

Thrown away after iteration 1	Kept for iteration 2
Hard-coded sample menus and prices inside templates	Template structure and the shared stylesheet `static/css/style.css` (the "polish" investors judge)
Fake totals typed into the cart page	URL design: `/restaurant/<id>/`, `/orders/cart/`, `/orders/checkout/`, `/orders/history/`
The unused `orders/success.html` page left from the first click-through (it is still in the repository and is no longer linked)	Data models and the seed command `seed_restaurants` (10 restaurants, 20 menu items)
	Test data and, from iteration 2, all automated tests

Throw-away code is allowed in a prototype only when it is thrown away. Leftovers such as `success.html` are the start of technical debt (see Task E).

Task B (Module 2): Agile planning with AI (Kanban)

Board. Columns: Backlog, Design, Build, Test, Done. WIP limit 3 on Design, Build and Test. Capacity used in the simulation: 1 designer, 2 builders, 1 tester (the 3 developers share the roles). Working days Monday to Friday starting Monday 5 October 2026. Cards are pulled only when the next column has space.

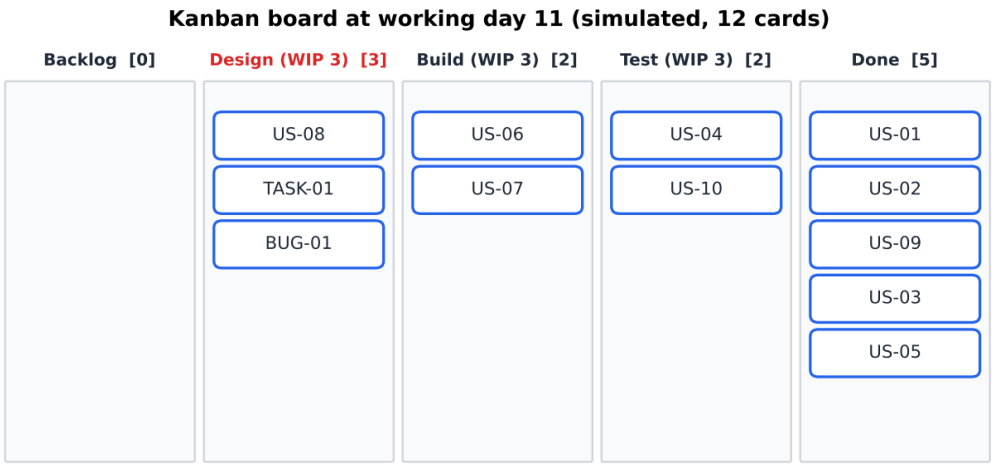


Figure 1: Kanban board on working day 11 of the simulation. Design is at its WIP limit of 3.

12 simulated cards with dates (generated by `labs/kanban/kanban_sim.py`; the dates are the day a card entered each column). *This is a simulation: the durations are assumptions, not measurements of a real team.*

ID	Card	Created	Design	Build	Test	Done	Lead (days)	Cycle (days)
US-01	Browse 10 restaurants	5 Oct	5 Oct	6 Oct	8 Oct	9 Oct	4	4
US-02	Restaurant menu	5 Oct	5 Oct	7 Oct	9 Oct	12 Oct	5	5
US-09	Register / sign in	5 Oct	5 Oct	8 Oct	13 Oct	14 Oct	7	7
US-03	Add item to cart	5 Oct	6 Oct	12 Oct	14 Oct	16 Oct	9	8
US-05	View cart and total	5 Oct	7 Oct	13 Oct	15 Oct	19 Oct	10	8
US-04	Change quantity (max 20)	5 Oct	8 Oct	14 Oct	19 Oct	20 Oct	11	8
US-10	Block unavailable items	5 Oct	12 Oct	15 Oct	19 Oct	22 Oct	13	8
US-06	Apply coupon	5 Oct	13 Oct	16 Oct	22 Oct	23 Oct	14	8
US-07	Checkout details	5 Oct	14 Oct	19 Oct	22 Oct	26 Oct	15	8
US-08	Place order and history	5 Oct	15 Oct	21 Oct	28 Oct	30 Oct	19	11
TASK-01	Jenkinsfile + Docker Compose	8 Oct	16 Oct	22 Oct	27 Oct	28 Oct	14	8
BUG-01	Checkout loses delivery details	15 Oct	19 Oct	23 Oct	29 Oct	2 Nov	12	10

Average lead time = 11.08 working days (lead time = Created to Done, sum 133 days over 12 cards). **Average cycle time = 7.75 working days** (Design to Done). Throughput = 3 cards per week. The last card (BUG-01, found on day 8) finishes on 2 November, one working day after the 4-week window ends on 30 October; Kanban shows this early, so the team would cut scope (for example TASK-01) or drop BUG-01's design step.

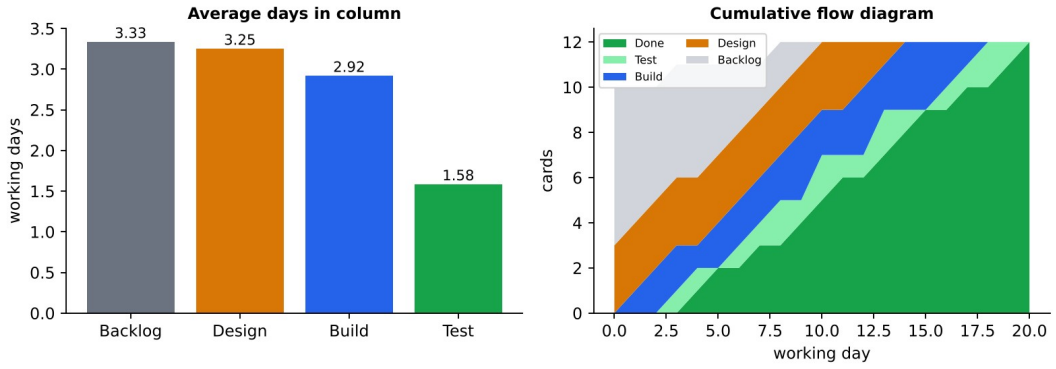


Figure 2: Average days a card spends in each column, and the cumulative flow diagram of the 12 cards.

Asking AI to guess the bottleneck from the data. I gave the dates (not my analysis) to a separate AI session and asked for the bottleneck. It answered: *Design, with Build the next constraint; Test is not a constraint.* It quoted Design 3.25 days against Build 2.9 and Test 1.5.

AI claim	My check against the data	Verdict
Design averages 3.25 days, Build 2.9, Test 1.5	My calculation: 3.25, 2.92, 1.58	Correct
Design was at its WIP limit of 3 from 8 to 12 Oct	Simulation: Design was at the limit on 12 days	Correct
Design took 4 days for "6 of the last 7 cards"	Only 4 of the last 7 cards took 4 days (US-04, US-08, TASK-01, BUG-01)	**Wrong** (overstated)
Build went to 4, 5, 3, 4 days for US-06, US-08, TASK-01, BUG-01	Matches the table	Correct
Test is not a constraint	Test had max 2 cards, never at the limit	Correct

Conclusion: the AI named the right column, but one number was invented, so a guess from an AI must be checked against the data. Note that "time in Design" includes waiting for a free Build slot, so the real cause is the single designer plus the Build limit. Actions: pair on Design and Build together, and stop pulling new cards into Design while two cards wait in Build.

Task C (Module 3): Development and deployment

- **Short feature branches and pull requests.** `main` always runs; work happens in branches such as `feature/coupon` or `bugfix/checkout-details`, no longer than two days; every merge is a pull request that needs one review and a green Jenkins build (strategy diagram in T3.3, real merge conflict in L3.3).
- **Add-to-cart API prompt with few-shot examples.** I wrote one working example (`remove_from_cart`) in the project's own style and asked for `add_to_cart` in the same style, with constraints (404 for missing or unavailable items, maximum 20, `@require_POST`, safe redirect) and a request for tests. Prompt, output and score are in L3.1 (version 4, score 4 of 5).
- **Jenkins builds on every push with a failure notification step.** `Jenkinsfile` has `triggers { githubPush() }`, four stages and a `post { failure { mail ... } }` block (listing in Appendix C; failed and passed build in L3.4).
- **Docker Compose runs the app and the database.** `docker-compose.yml` starts a Django `web` service and a PostgreSQL `db` service with a health check (L3.5).

Task D (Module 4): Testing and monitoring

Equivalence classes for the quantity field (the quantity of one item in the cart; valid range 1 to 20, `MAX_QTY`):

Class	Range	Representative value	Expected	Test
Valid, lower boundary	1	1	Cart line with quantity 1	EC-Q1
Valid, typical	2 to 19	5	Quantity 5	EC-Q2
Valid, upper boundary	20	20	Quantity 20	EC-Q3
Invalid, above range	21 and more	21	Stays at 20 (capped)	EC-Q4
Invalid, below range	0 or negative	0	Line removed, never negative	EC-Q5

Equivalence classes for the coupon field (SAVE10 = 10 % off, FLAT5 = $5 off; subtotal of two cheeseburgers = $17.98):

Class	Example input	Expected total	Test
Valid percentage code	`SAVE10`	$16.18	EC-C1
Valid flat code	`FLAT5`	$12.98	EC-C2
Valid, different case or spaces	`save10`	$16.18	EC-C3
Invalid unknown code	`FREEFOOD`	$17.98 (no discount, error shown)	EC-C4
Invalid empty code	(empty)	$17.98	EC-C5
Boundary: discount larger than subtotal	`FLAT5` on a $1.50 cart	$0.00, never negative	EC-C6
Stored value	`SAVE10` then order	Order total saved as $16.18	EC-C7

Results: on the prototype **as submitted**, 6 of these 12 tests fail because the coupon and the quantity cap do not exist yet (FR-06 is not built); on the fixed copy all 12 pass (`Ran 12 tests ... OK`, log in L4.4).

Selenium: add 3 items, apply coupon, place order. Two versions were run in headless Chromium 153: a plain script and a Page Object version (code and comparison in L4.3). Both pass: three menu items are added (subtotal $14.48), `SAVE10` gives a total of $13.03, the order is placed and saved with 3 items.

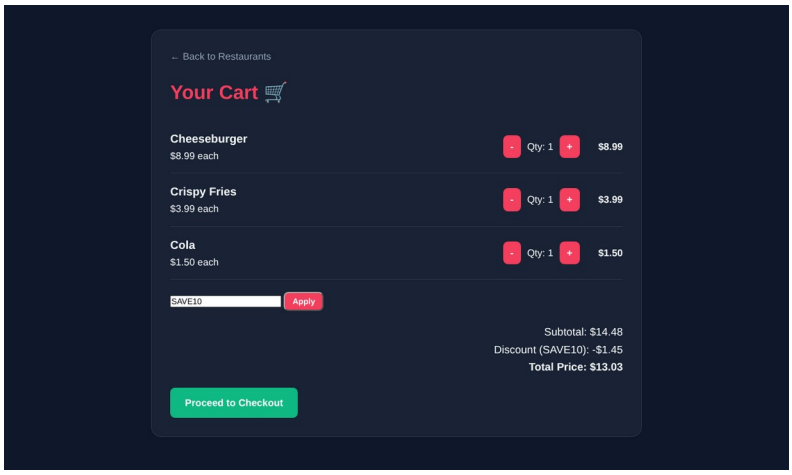


Figure 3: Selenium run, cart page after applying SAVE10 (total $13.03).

One manual exploratory session on the menu page. Charter: *explore `/restaurant/<id>/` on a phone-sized screen to find usability and data problems.* The session was run as a script in Chromium at 375 px width on the prototype as submitted.

Check	Observation
All 10 menu pages open	Yes, status 200 for all 10 restaurants
Items per menu	2 items per restaurant (20 in total); a real menu would look empty
Horizontal scrolling at 375 px	None
Images without `alt` text	0 of 3
Feedback after pressing “+ Add”	None: no message, and the “Cart 🛒” link shows no count (**UX-01**)
“+ Add” is a link (GET)	`/orders/cart/add/21/` changes the cart on a GET request (**DEF-07**)
Add the same item 4 times	Cart shows quantity 4 correctly
Keyboard Tab	Focus goes to a link; focus is usable

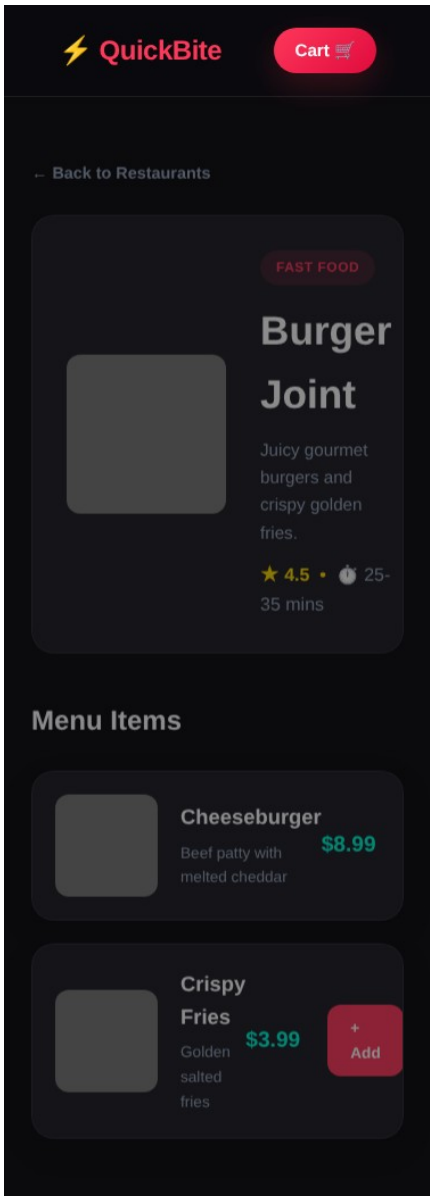


Figure 4: Menu page at 375 px width during the exploratory session.

Self-healing locators after a UI change. A Selenium test breaks when the page changes, for example when the id `apply-coupon` is renamed. A self-healing tool stores several attributes for each element (id, name, text, CSS path, position). When the main locator fails, it looks for the best match by the other attributes, uses it, and records the change so that a human can approve it. For Food Delivery MVP this helps when the designer renames CSS classes, because the UI changes often during the 4 weeks. The risks are that a wrong match can click the wrong button ("Apply" instead of "Checkout") and the test still passes, and that the tool hides real UI regressions. My rule: use self-healing only for navigation steps, never for the final assertion on the total or the order record, and review every healed locator in the pull request. Stable test ids (`data-testid`) and the Page Object Model (L4.3) are cheaper than self-healing and are used first.

Task E (Module 5): Quality and ethics

Usability target: order in 4 taps (NFR-01). Measured with a browser script on the fixed copy (signed-in user, one item): tap 1 "+ Add", tap 2 "Cart", tap 3 "Proceed to Checkout", tap 4 "Place Order". The result is **4 taps**, which meets the target; typing name, phone and address is not a tap. On the prototype as submitted, a new user cannot reach tap 1 at all: the home page returns an error for visitors who are not signed in (DEF-01) and the sign-in page returns an error (DEF-02).

Efficiency target: pages load in under 2 seconds (NFR-02). Measured with Chromium navigation timing, 10 loads per page, `loadEventEnd`:

Page	Average (ms)	Slowest of 10 (ms)	Target	Result
Home (restaurant list)	123.6	129.3	< 2000	Pass
Menu	122.8	131.2	< 2000	Pass
Cart	121.1	128.4	< 2000	Pass
Checkout	119.0	125.1	< 2000	Pass

> **Evidence note.** These are local-server times. The food photos come from `images.unsplash.com` (30 hot-linked URLs in the seed file) and the sandbox cannot reach that site, so the photo requests were answered with a 1-pixel placeholder. On a real phone network the photos decide the load time, so the 2-second claim must be re-measured on the investor's network, and photos should be resized and stored with the app.

SonarQube and technical debt. SonarQube was not available. Equivalent checks on the prototype (`orders/views.py`, 81 code lines): duplicated lines 4.9 % with a 4-line window and 8.6 % with a 3-line window; the cart-reading loop is copied in `view_cart` and `checkout`; the expression that reads a quantity (`val.get('quantity', 0) if isinstance(val, dict) else int(val)`) is copied **4 times**; pylint score 7.56 of 10 with 36 messages (mostly trailing whitespace and an unused import); radon: `checkout` complexity B (9), average A (3.3); bandit: 1 low finding (hard-coded `SECRET_KEY`). After the refactor the duplication is 0 % and the score 8.76 (L5.2).

Ethics: AI-generated food photos and fake reviews.

Risk in Food Delivery MVP	What the code does today	My rule
AI-generated or stock food photos shown as the real dish	30 Unsplash photos are hot-linked as the real menu photos	Label sample photos "illustrative" or use photos supplied by each restaurant; never show an AI image as the real dish. Check the photo licence before launch
Fake reviews and ratings	`Restaurant.rating` is a stored number (default 4.0) that the seed data filled in; no real customer produced it	Show ratings only when they come from real orders. In the demo, mark them "sample data". Never write or buy reviews with AI
Misleading investors	A polished demo can hide DEF-04 (orders without delivery details)	Tell investors what is real and what is sample data
Personal data	Orders store name, phone and address	No real customer data in the repository or in AI prompts (L5.5)

Part 2: Theory Questions

Module 1: Software Engineering, SDLC and AI Tools

T1.1 [Basic] Phases of the SDLC and one deliverable for Food Delivery MVP

The SDLC is the ordered set of activities that takes software from an idea to a running, maintained product.

Phase	What happens	One deliverable for Food Delivery MVP
1 Planning	Decide scope, time, team, risks	4-week plan for 3 developers and a Kanban board with 12 cards
2 Requirements analysis	Find out what users and investors need	Requirement list FR-01 to FR-10, NFR-01 to NFR-03 (Table 1)
3 Design	Decide structure and screens	Data model (Restaurant, FoodItem, Order, OrderItem), URL list, clickable screens
4 Implementation	Write the code	Django app with session cart and order placement
5 Testing	Check the code against requirements	10 manual cases (TC-01 to TC-10) and automated tests
6 Deployment	Make it available	`Dockerfile`, `docker-compose.yml`, Jenkins build artifact
7 Maintenance	Fix and improve after release	Defect list (DEF-01 to DEF-07) and backlog for version 2

In a prototype project these phases are repeated in small loops (T1.2) instead of being run once.

T1.2 [Basic] The Prototype model

In the Prototype model the team builds a quick, incomplete version early, lets users try it, learns what is wrong or missing, and then either improves the prototype or rebuilds it properly.

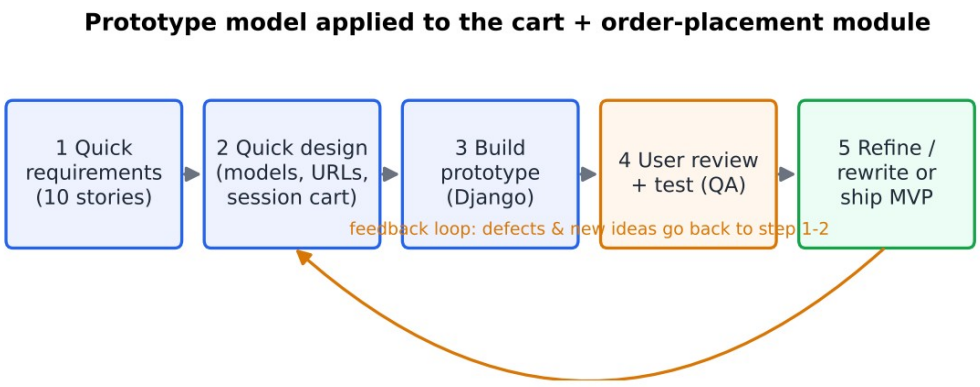


Figure 5: The Prototype model applied to the cart and order-placement module.

Two reasons it fits Food Delivery MVP. (1) Investors judge “speed of ordering and how polished it feels”, which nobody can describe in a document; they have to see and click it, so an early clickable prototype (Task A, iteration 1) answers the real question. (2) Requirements are unclear: the client notes (L1.4) do not say how payment or discounts work, and 4 weeks leaves no time for a long specification, so learning from users early is cheaper than guessing.

Two situations in which I would change the model. (1) If real payments and customer data were added, a mistake would cost money and trust, so I would move to a more controlled model such as Incremental with a formal test phase or the V-Model. (2) If the team grew from 3 developers to several teams with shared components, throw-away code would become too expensive and I would switch to Incremental or Scrum with architecture agreed up front.

T1.3 [Applied] Which phases gain most from AI, and which stay human-led

Phase	AI gain	Main risk	Decision
Requirements	Medium: turns messy notes into a list quickly	Invented requirements (in L1.4, 3 of 12 AI items were invented)	AI drafts, human decides
Design	Low to medium: suggests a data model	Looks reasonable but may ignore the team's constraints	Human-led
Implementation	**High**: boilerplate such as the cart view	Insecure or wrong code (the first prompt in L3.1 gave a wrong model name and stored prices in the session)	AI writes, human reviews
Testing	**High**: many test cases fast; test data	Wrong tests: 4 of 12 AI tests in L4.4 asserted things nobody asked for	AI proposes, human filters
Deployment	Medium: Dockerfile and Jenkinsfile drafts	Secrets in files; wrong settings	Human reviews every file
Maintenance	Medium: explains old code	Wrong explanation	Human-led
Planning and estimation	Low: AI does not know the team	Over-confident numbers (L2.4: AI gave 8 points to order placement)	Human-led
Ethics and accountability	None	Responsibility cannot be handed to a tool	Human only

The phases that must stay human-led are requirements decisions (what the investors really want), design trade-offs, final code review and release approval.

T1.4 [Applied] GitHub Copilot, Tabnine and ChatGPT

Aspect	GitHub Copilot	Tabnine	ChatGPT
Type	Code completion in the editor (also chat)	Code completion in the editor	Chat assistant in a browser or app
Features	Completes lines and functions from the open files; chat inside the IDE	Completion; can be set to learn from the team's own code	Explains, writes whole files, tests, documents, reviews
Context it sees	The open project files	The open project files	Only what the user pastes
Privacy	Code snippets go to the vendor's cloud; business plans offer stricter terms	Offers private or on-premise deployment, the strongest option for private code	Everything typed in is sent to the vendor; pasting code or customer data is a risk
Integration	VS Code, JetBrains, GitHub	Many IDEs	Browser, no project integration
Best for	Daily coding	Teams with strict privacy	Planning, test ideas, explanations

Vendor privacy and plan terms change often; check each vendor's current policy before choosing.

Recommendation for Food Delivery MVP: use ChatGPT (the tool the case names) for requirements, test ideas and explanations, and add a completion tool such as Copilot for typing the repetitive Django code. Rules for both: no real customer data, no secrets, and every output goes through review and tests, as the evidence in L1.3 and L3.1 shows that AI code can look right and still be wrong for the project.

T1.5 [Advanced] Validating AI-written requirements and AI-generated test cases

Requirements. Step 1: trace every AI requirement to a source (a line of the client's notes or a scenario number); anything without a source is marked *invented*. Step 2: check that it is testable, with a number or an observable result; otherwise it is *vague*. Step 3: ask the client to confirm the vague and invented items. Step 4: give a requirement ID only after that. In L1.4, 12 AI items were classified this way: 5 correct, 4 vague, 3 invented.

Test cases. Step 1: derive the expected result from the requirement, not from the code, so that a test of an invented feature is visible (in L4.4, the AI wrote tests for 5 % tax, for a JSON cart and for an "out of stock" message, none of which is a requirement). Step 2: run each test against the code and against a deliberate break

(a mutation: change `+1` to `+2` and see the test fail, as done in L3.2). Step 3: for each failing test decide: is the code wrong (defect) or the test wrong (hallucination)? Step 4: compare coverage with the requirement list to find missing tests. The accuracy rate of AI tests is then measured (L4.4: 8 of 12, 66.7 %).

Module 2: Agile Practices and AI-Driven Planning

T2.1 [Basic] Why the Agile Manifesto was created, and two values for this team

In 2001 a group of practitioners wrote the Manifesto because heavy, plan-driven processes delivered late, over budget and not what users needed. It prefers (1) individuals and interactions over processes and tools, (2) working software over comprehensive documentation, (3) customer collaboration over contract negotiation, (4) responding to change over following a plan.

- **Working software over comprehensive documentation.** The 3 developers do not write a long specification. At the end of every week there must be something that runs and can be shown to a test user, such as the clickable screens in week 2. This is also why the repository holds tests and a one-page README instead of a design book.
- **Responding to change over following a plan.** The test users' answers (Task A) will change the plan, for example by adding the coupon box. The Kanban board lets the team swap the next card without waiting for a new plan (BUG-01 entered on day 8 in Task B).

T2.2 [Basic] Scrum, XP, Lean and Kanban compared

	Scrum	XP	Lean	Kanban
Core idea	Fixed-length sprints with roles	Engineering practices (pair programming, TDD, small releases)	Remove waste, deliver fast	Visualise work, limit work in progress
Time box	Sprint, 1 to 4 weeks	Iteration, 1 to 2 weeks	None	None; continuous flow
Roles	Product Owner, Scrum Master, team	Coach, customer, programmers	None	None required
Change during iteration	Avoided until the next sprint	Welcomed	Welcomed	Welcomed at any time (if WIP allows)
Key measures	Velocity	Test coverage, build health	Waste, lead time	Lead time, cycle time, WIP
Best for	Stable teams with a backlog	Teams that need very high code quality	Reducing process waste	Support, fixes, small teams with a changing order of work

Why Kanban for Food Delivery MVP. The team is only 3 people, the work arrives in an unpredictable order (the investor feedback in week 3 can change priorities and bugs such as BUG-01 appear at any time), and there are only 4 weeks, so there is no room for sprint ceremonies. A WIP limit of 3 matches the team size, and lead time shows quickly whether the 4-week goal is at risk (Task B showed a finish one day late).

T2.3 [Applied] The sprint cycle and a sample sprint goal

Scrum (the sprint cycle is the standard Scrum timebox for a 2-week sprint; Food Delivery MVP itself uses Kanban, so I use this as a short iteration):

Event	Timebox (2-week sprint)	Purpose
Sprint planning	up to 4 hours	Choose the sprint goal and the backlog items
Daily scrum	15 minutes every day	Share progress and blockers; adjust the day's plan
Development	2 weeks	Build and test the selected items
Sprint review	up to 2 hours	Show the working increment to stakeholders (investors)

Event	Timebox (2-week sprint)	Purpose
Sprint retrospective	up to 1.5 hours	The team decides how to work better next sprint

Sample sprint goal: *By the end of the sprint a signed-in user can add items from any of the 10 restaurants to a cart and place an order whose delivery details and total are saved and visible in order history (US-03, US-05, US-07, US-08).*

T2.4 [Applied] How AI can forecast the delivery date from past data

The forecast uses past data: for each finished card its lead time or story points per week (velocity). A simple method is a Monte Carlo simulation: pick random past weekly throughputs many times, add them until the remaining work is done, and read the date by which 85 % of the runs finish. In this project 12 cards finished in 4 weeks at 3 cards per week (Task B), so 8 remaining cards would be forecast at about 2.7 weeks.

Assumptions: the future looks like the past (same people, same card size); cards are of similar size; the data is complete and recorded honestly; no big scope change. **Risks:** 3 developers and only 12 cards give very little data, so the forecast is a wide range, not a date; one illness changes everything; AI can sound certain about a number that has no evidence (in L2.4 the AI gave a total of 38 points with no team data); and a forecast that people treat as a promise causes pressure. Use the forecast as a range, update it weekly and keep the human estimate next to it.

T2.5 [Advanced] ChatGPT, Copilot and Jira AI in an Agile team

- **ChatGPT** writes user stories and acceptance criteria from notes, suggests estimates (L2.4), drafts retrospective summaries and test ideas.
- **Copilot** speeds up coding of repetitive code, can explain pull-request changes and suggests tests inside the IDE.
- **Jira AI features** (for example summarising issues, suggesting issue fields, searching with natural language) help keep the board tidy and find duplicate cards. *Check the exact features of the Jira plan in use.*

Three benefits for Food Delivery MVP: (1) faster drafting: the 10 stories and 12 cards were drafted in minutes; (2) a second opinion on estimates: the AI's 8 points for US-08 pointed to a real risk, the missing transaction (L5.3 R1); (3) more test ideas: the AI found the empty-cart and unavailable-item cases.

Three challenges: (1) wrong or invented content, for example 3 of 12 invented requirements (L1.4) and one wrong number in the AI's bottleneck answer (Task B); (2) privacy, because notes and code sent to a cloud tool may contain client data; (3) over-trust: a 3-person team can stop thinking critically and plan around AI numbers (L2.4).

Module 3: Development and Deployment Practices using AI Tools

T3.1 [Basic] Code completion tools versus chat tools for cart and order placement

A completion tool (Copilot, Tabnine) lives in the editor and proposes the next lines using the files that are open. When I type `def add_to_cart(request, item_id):` it can complete the body in the style of the `remove_from_cart` function that is already in the file, so it uses the project's real names (`FoodItem`, `view_cart`). A chat tool (ChatGPT) works only on what I paste, answers in a conversation and can do larger jobs, such as writing the whole checkout view, explaining a bug or producing 10 test cases, but it does not see the project, so it may invent names. In L1.3 and L3.1 the chat answers used `MenuItem`, `Product` and a URL called `cart` where the project has `FoodItem` and `view_cart`. Rule of thumb: completion for small, repetitive, in-style code; chat for design questions, tests and reviews; both need review.

T3.2 [Applied] Refining a prompt that returned insecure code

The first prompt in L3.1 (“write add to cart in django”) returned a generic cart class that stored the price in the session and read the quantity from the user’s POST data; a later version redirected to an unchecked `Referer`. To refine: (1) state the context (Django 6, models, URL names), (2) state the cart format, (3) list security requirements in the prompt: validate the item id with `get_object_or_404(..., is_available=True)`, cap the quantity, never take prices from the browser, change state only on POST with CSRF, redirect only to a URL on the same host, (4) include a working example in the project’s style (few-shot), (5) ask for unit tests in the same answer.

Review I would add: a checklist (input validation, authorisation, CSRF, open redirect, price from the database, transaction around order creation); `bandit` and pylint in the Jenkins build; a human review of every AI pull request; and tests that try bad input. In L5.3 this review found a real open redirect and an order saved without a transaction.

T3.3 [Basic] Repository, commit, branch, merge, pull request and conflict

- **Repository:** the folder with all project files and their full history (the GitHub repository `FoodDeliveryMVP`).
- **Commit:** a saved snapshot with a message (for example “Fix checkout total”).
- **Branch:** a separate line of commits, so that one feature can be developed without breaking `main`.
- **Merge:** joining the commits of one branch into another.
- **Pull request (PR):** a request on GitHub to merge a branch, with a place for review and automatic checks.
- **Conflict:** git cannot decide because two branches changed the same lines; a person must choose (shown in L3.3).

Branching strategy for 3 developers in 4 weeks: a simple flow with short-lived branches.

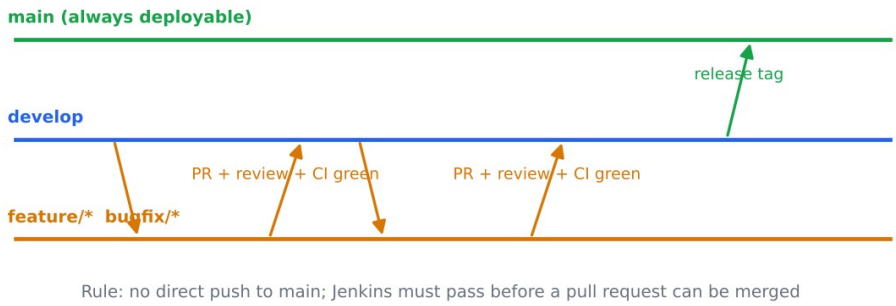


Figure 6: Branching strategy: short-lived feature branches merged by pull request when review and Jenkins are green.

`main` holds the demo version for investors; `develop` collects finished work; branches are named `feature/...` or `bugfix/...`, last at most two days, and are merged by PR. Each developer takes cards from different columns of the board so that the same file is rarely edited by two people.

T3.4 [Applied] Jenkins jobs, pipelines, the Jenkinsfile, triggers and agents

- **Job:** one configured task that Jenkins can run (for example “build Food Delivery MVP”).
- **Pipeline:** a job described as stages that run in order (Checkout, Build, Test, Package).
- **Jenkinsfile:** the text file in the repository that defines the pipeline, so the build is versioned with the code.
- **Trigger:** what starts a build; here `githubPush()` starts it on every push.
- **Agent:** the machine or container where the stages run (`agent any` uses any free one).

How a failed test stops a release. Each stage runs a shell command. If the Test stage command returns a non-zero exit code, Jenkins marks the stage failed, **skips the later Package stage** (so no release zip is produced), and runs the `post { failure { ... } }` block, which e-mails the team. A branch rule on GitHub that requires the

Jenkins check to be green blocks the merge. L3.4 shows this: build #1 failed with 17 of 35 tests failing and no package, and build #2 passed after the fixes.

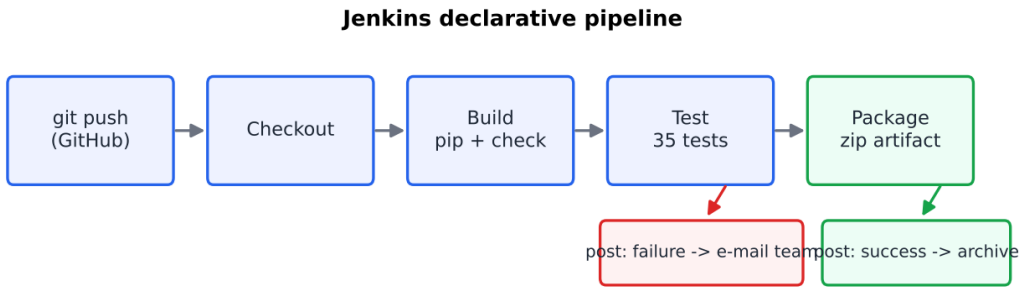


Figure 7: The pipeline used for Food Delivery MVP.

T3.5 [Advanced] Benefits and challenges of AI-assisted development and deployment (CI/CD and containers)

Benefits for Food Delivery MVP	Challenges for Food Delivery MVP
AI writes a first `Dockerfile`, `docker-compose.yml` and `Jenkinsfile` in minutes	AI files can contain wrong settings or secrets; the compose file in this project needed `POSTGRES_PASSWORD` to come from the environment, not from the file
CI finds problems on every push: 17 failing tests were visible at once in build #1	CI only finds what the tests cover; an AI-generated suite with wrong tests gives a false green or false red (L4.4)
Containers give the same app and database for all 3 developers and the investor demo ("works on my machine" disappears)	Container and pipeline skills are new for the team; a broken compose setup can cost days in a 4-week project
AI-assisted fixes are quick (the fix for DEF-03 took a few lines)	AI code can be insecure (open redirect, state change on GET) and must be reviewed
Pipeline runs the same checks every time	Build time and server cost grow; Selenium needs a browser in the build agent

Module 4: AI-Based Testing and Monitoring

T4.1 [Basic] Software testing; manual versus automated; what to automate

Software testing is running the software with chosen inputs to compare the actual behaviour with the expected one and to find defects before users do.

	Manual	Automated
Who or what	A person follows or explores	A script or test runner
Strength	Finds usability and "looks wrong" problems; no set-up cost	Fast, repeatable, runs on every push
Weakness	Slow, varies between people	Must be written and maintained; cannot judge looks

Automate the stable, repeated, high-risk checks: cart arithmetic (AI-TC-05), order saving (AI-TC-08 to 10), coupon classes (EC-C1 to C7), the end-to-end order with Selenium, because they run on every Jenkins build. **Keep manual** the exploratory session on the menu page, the "does it feel polished" check for investors, and a first look at any new screen.

T4.2 [Basic] The test pyramid for Food Delivery MVP

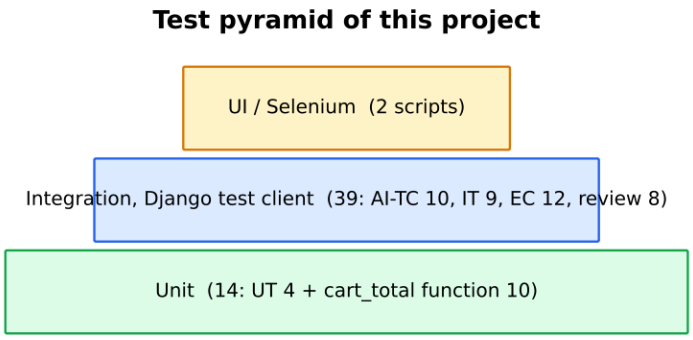


Figure 8: Test pyramid with the real counts of this project.

Level	What is tested	Who writes and runs	Count here
Unit (many, fast)	Single functions: `_qty`, discount rules, `cart_total`	Developer, on every save	14
Integration	Views with database and session through the Django test client: add item, cart total, checkout saves an order	Developer and tester, on every push	39
UI / end-to-end (few, slow)	The whole order in a real browser	Tester, on every push or nightly	2 scripts (6 runs)
Manual / exploratory	Look, feel and surprises	Tester and the product owner	10 cases and 1 session

T4.3 [Applied] Page Object Model and Selenium maintenance

The Page Object Model (POM) puts the locators and actions of each page into one class (`MenuPage`, `CartPage`, `CheckoutPage`), and the tests only call methods such as `cart.apply_coupon('SAVE10')`. If the designer changes the id of the “Apply” button, only `CartPage` changes. In L4.3 the plain script repeats 33 locators across 3 tests and refers to the coupon button in 3 places; the POM version holds each locator once (10 in total) and the button in 1 place, so a UI change costs 1 edit instead of 3, and the tests read like the user story.

T4.4 [Applied] AI-assisted test generation and test prioritisation before a release

AI test generation means giving the AI the requirement or the code and asking for test cases, test data and edge cases. It is fast and finds cases people forget, but it also invents tests, so each test is checked against the requirement (L4.4: 8 valid of 12, 66.7 %).

AI test prioritisation orders the tests so that the ones most likely to fail run first, using history (which tests failed recently), what code changed, and risk. For the release of Food Delivery MVP I would order the tests by **risk = impact × likelihood**:

Priority	Tests	Reason
1	Order saves items, details and total (AI-TC-08 to 10, TC-10), sign-in and checkout access (TC-07)	Money and data; DEF-01, DEF-02 and DEF-04 were found here
2	Cart arithmetic and coupon (AI-TC-05, EC-C1 to C7)	Wrong totals destroy trust
3	Unavailable items and quantity limits (DEF-05, EC-Q)	Wrong orders
4	Selenium end-to-end order	Catches integration breaks
5	Look and feel checks, exploratory session	Matters for investors, rarely blocks an order

T4.5 [Advanced] Defect prediction and AI-based bug detection

Defect prediction trains a model on past project data to estimate which files or changes are most likely to contain defects, so the team tests them first. AI-based bug detection reads code and flags suspicious patterns, such as an order that is created without a transaction.

Data I would collect for Food Delivery MVP: per file and per commit: lines changed, number of authors, past defect count (DEF IDs per file), cyclomatic complexity (radon: `checkout` is the most complex at B 9), test coverage (93 % overall, 90 % for `orders/views.py`), review comments, and, from production, errors per URL. With a single commit and 7 defects there is not enough data to train a model yet, so I would start with simple rules: files with high complexity and many changes get extra review.

How the team acts on predictions: put the top-risk files (here `orders/views.py` and the checkout template) first in review and in the Jenkins test order; add tests before changing them; check each prediction on the next defects to see if it was right; and never use a prediction to judge a developer.

Module 5: AI-Powered QA and Ethical Development

T5.1 [Basic] Two quality attributes that matter most, and how to measure them

Attribute	Why it matters most	How I measure it	Result
Efficiency (performance)	Investors judge speed of ordering (scenario)	Page load by browser timing (NFR-02, < 2 s); taps to order (NFR-01, 4)	119 to 124 ms locally; 4 taps (Task E)
Reliability / correctness of orders	An order with no address or $0.00 total cannot be delivered or paid	Share of orders with complete details and a correct total; test pass rate	Prototype: 2 of 2 orders in the monitored database incomplete (monitor in L4.5); fixed copy: 35 of 35 tests pass

Usability (taps to order, feedback after add) is third and also tested (Task D).

T5.2 [Applied] SonarQube and its terms in this project

SonarQube is a server that analyses source code on every push and reports quality problems; the project passes or fails a **quality gate**.

Term	Meaning	Example in Food Delivery MVP
Bug	Code that is wrong and will cause a failure	Ignoring `full_name`, `phone`, `address` and total when saving an order (DEF-04)
Vulnerability	A weakness an attacker could use	Redirecting to an unchecked `Referer` (open redirect, DEF-07)
Security hotspot	Code that needs a human security check	Hard-coded `SECRET_KEY` and `DEBUG = True` in `settings.py` (bandit B105)
Code smell	Maintainability problem	The quantity expression copied 4 times; unused import `get_object_or_404`; trailing whitespace
Duplication	Copy-pasted code	The cart-reading loop in `view_cart` and `checkout` (4.9 % duplicated lines)
Coverage	Share of lines executed by tests	93 % overall; 90 % for `orders/views.py`
Quality gate	Pass/fail rule on new code	Example: no new bugs, coverage of new code at least 80 %, duplication under 3 %

T5.3 [Applied] AI code review, human review and static analysis

	Static analysis (pylint, bandit, SonarQube)	AI review	Human review
Finds	Known patterns: unused imports, hard-coded key, complexity	Logic, design and security ideas in plain language	Business meaning, whether it is the right thing to build
Misses	Logic errors such as DEF-04 (valid code that ignores a form field)	Can be wrong (2 of the AI's 8 review suggestions were false alarms in L5.3)	Gets tired; slower
Consistency	Same every time	Varies by prompt	Varies by person
Cost	Cheap, automatic	Cheap, fast	Most expensive

Combination: static analysis runs automatically in Jenkins (rule-based findings, no argument); the AI reviews each pull request and every suggestion is tested before it is accepted (L5.3); a human reviewer makes the final decision and checks the business meaning. Evidence: the tests I wrote for the AI's review suggestions confirmed 5 as real defects, showed 2 to be false alarms, and left 2 as style opinions.

T5.4 [Advanced] Accountability for harm caused by AI-generated code

- **Developer:** the developer who accepts, merges and ships the code is accountable for it. "The AI wrote it" is not a defence, because the developer chose to use it without enough review. In L5.3 the AI-generated checkout had no transaction; a developer who merged it unreviewed would own the orphan orders.
- **Company (the start-up):** responsible for setting the policy (L5.5), training, review rules and for what is shown to investors and customers; it carries the legal responsibility toward customers and for data protection.
- **Tool vendor:** responsible for how it builds and describes its tool, security of its service and the terms it offers; but vendor terms normally say that the user must check the output, so liability mostly stays with the user and the company. *The exact terms differ by vendor and contract and should be read; I am not giving legal advice.*

A harm in Food Delivery MVP could be orders delivered with no address (DEF-04), or a customer's phone number leaked through a debug page (`DEBUG = True` shows variables). The chain is: the developer should have tested and reviewed, the company should have required it, and the vendor is a supplier whose output is an input to be checked.

T5.5 [Advanced] Benefits and challenges of AI in SQA, and why AI help must be acknowledged

Benefits (with evidence from this report)	Challenges (with evidence)
Many test cases quickly: 10 AI test cases found 2 real defects (L1.5)	Hallucinated tests: 4 of 12 raw AI tests were wrong (L4.4)
Review suggestions that found a real open redirect and missing transaction (L5.3)	2 of 8 review suggestions were false positives (L5.3 R7, R8)
The AI function passed 10 of 10 edge-case tests, my hand-written one 7 of 10 (L5.4)	AI explanation of data can contain a wrong number (Task B)
Fast refactoring with no change in behaviour (L3.2)	Privacy and bias: data pasted into tools; AI trained on code with bad habits

Why AI assistance must be acknowledged: (1) honesty: the reader should know who did the work and judge the work correctly; (2) accountability: it shows who checked the output and how (Appendix A); (3) integrity and law: AI output can contain code under a licence or text that is not the student's own, and universities and employers require disclosure; (4) reproducibility: others can repeat and audit the work. Hiding AI use makes a failure look like the student's hidden incompetence or dishonesty, and removes the reader's chance to check the AI-written parts more carefully.

Part 3: Lab Questions

All code, tests and logs are in the repository folder `case03/`. The "prototype as submitted" is the code in `FoodDeliveryMVP.zip`; the "fixed copy" is the same code with the corrections listed in L4.1 and L5.3 plus the coupon feature (FR-06).

Module 1 Labs: SDLC, Process Models and AI Tools

L1.1 [Basic] SDLC phases of Food Delivery MVP with one output per phase

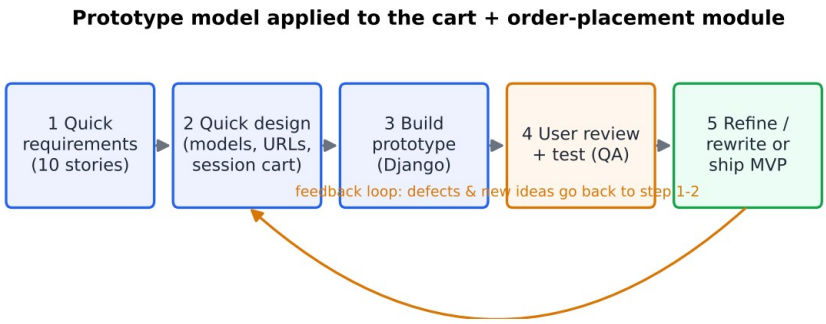


Figure 9: SDLC phases with the Prototype loop, one output per phase.

Phase	Output for Food Delivery MVP	Where it is in this report
Planning	4-week plan, Kanban board, 12 cards	Task B
Requirements	Table 1 below	Table 1
Design	Models, URLs, screens	T1.1, Task A
Implementation	Cart and order-placement code	repository
Testing	TC-01 to TC-10, AI-TC-01 to 10, EC tests	L4.1, L1.5, Task D
Deployment	Jenkinsfile, Dockerfile, compose	L3.4, L3.5
Maintenance	DEF-01 to DEF-07 and v2 backlog	L4.1

Table 1: Requirements used in every part

ID	Requirement	Story
FR-01	The user can see the list of 10 restaurants	US-01
FR-02	The user can open a restaurant and see its menu with prices	US-02
FR-03	The user can add a menu item to the cart with one tap	US-03
FR-04	The user can increase or decrease the quantity of an item (1 to 20)	US-04
FR-05	The cart shows line subtotals and a total	US-05
FR-06	The user can apply a coupon code (SAVE10 = 10 %, FLAT5 = $5)	US-06
FR-07	A signed-in user enters name, phone and delivery address at checkout	US-07
FR-08	Placing the order saves items, prices, delivery details and total, and clears the cart	US-08
FR-09	The user can see their past orders	US-08
FR-10	A visitor can register and sign in; unavailable or unknown items cannot be ordered	US-09, US-10
NFR-01	A user can place an order in 4 taps	
NFR-02	Pages load in under 2 seconds	

ID	Requirement	Story
NFR-03	What the user typed at checkout must be what is stored (data integrity)	

L1.2 [Applied] First milestone under Waterfall and under Incremental

Milestone: *a user can order from one restaurant.*

	Waterfall	Incremental
Plan	Complete requirements, then design, then code, then test, in order	Deliver 3 increments: (1) browse and menu, (2) cart, (3) checkout and order
Effort	Big up-front effort in requirements and design (about 5 days of 20) before any code	Effort spread over the 4 weeks; each increment includes its own tests
Time to first working screen	Week 3	End of week 1
Risk	High: wrong requirements are found only at the end (e.g. the missing coupon box or the blank total)	Lower: each increment is tried by users
Feedback	Only after testing at the end	After every increment (3 times)
Cost of change	High	Low to medium
Fit with the scenario	Poor: investors judge look and speed and need an early demo	Good, and the Prototype model (T1.2) is a form of it

L1.3 [Applied] The same prompt in two AI tools

Prompt (identical): *Write a Django 6 view add_to_cart(request, item_id) ... stores the cart in request.session['cart'] as {str(item_id): quantity}, then a checkout(request) that creates an Order (user, full_name, phone_number, address, total_amount) and OrderItem rows from the cart, clears the cart and redirects to 'order_history'. Models: restaurants.FoodItem, orders.Order, orders.OrderItem.*

> **Evidence note.** Only one vendor's assistant was available to me, so the two "tools" are two models of the same vendor: **Tool A = Claude Haiku 5.5**, **Tool B = Claude Sonnet 5.5**. If the module requires two different vendors (for example ChatGPT and Copilot), repeat this prompt there and replace the columns; the method of comparison is the same.

Criterion	Tool A (Haiku)	Tool B (Sonnet)
`add_to_cart`	`@require_POST`, 404 if unavailable, **quantity read from user input** (up to 99)	`@require_POST`, 404 if unavailable, always +1
Redirect after add	`redirect("cart")` (URL name does not exist in the project; it is `view_cart`)	`redirect(request.POST.get('next') or 'order_history')`: **unchecked user-supplied target (open redirect)**
`checkout`	`@require_POST` on the whole view, so the checkout **form page cannot be shown**	Handles GET (shows the form) and POST
Prices and total	From the database; `Decimal` sum	From the database; `Decimal` sum
Transaction	`transaction.atomic()`, `bulk_create`	`transaction.atomic()`, `bulk_create`
Unavailable items at checkout	Rejects the order with a message	Silently drops them
Explanation	3 lines, mentions the assumed URL name	3 lines, mentions the assumed template
Works in the project as is?	No (URL name, GET-based "+ Add" links in templates, form page)	Closer, but needs a safe redirect and CSRF forms

Criterion	Tool A (Haiku)	Tool B (Sonnet)
Privacy	Prompt contained no personal data. Both are from the same vendor, so the same terms apply	Same

Method: static review of both outputs against the project's URL names, models and templates; the outputs were not executed. **Result:** Tool B was more correct for the checkout flow (it serves the form); Tool A was safer for the cart redirect but allowed a client-chosen quantity and broke the form page. Neither output was usable without edits, which is the point of L3.1.

L1.4 [Applied] Eight messy client notes and the AI extraction

Notes (8 lines):

1. "we need the app to be super fast, investors hate waiting"
2. "people pick food from 10 restaurants, maybe more later"
3. "cart should remember stuff even if they refresh"
4. "has to have discount codes like SAVE10, marketing wants it"
5. "payment? card and UPI I think, ask Raj"
6. "must look premium, like Uber Eats but better"
7. "nobody should be able to order sold-out dishes"
8. "orders need a delivery address, name, phone obviously"

AI output (12 requirements) and my classification:

#	AI requirement	From note	Class	Corrected requirement / action
R1	Pages shall load in under 2 seconds	1	**Vague** (the number 2 s was added by the AI; it comes from the scenario)	NFR-02, confirmed by scenario
R2	The system shall list 10 restaurants	2	Correct	FR-01
R3	The system shall be scalable to more restaurants	2	**Vague** ("maybe more later"; no number)	Keep 10 for MVP; ask for the next target
R4	The cart shall persist across page refreshes	3	Correct	FR-03 / FR-05 (session cart)
R5	The user can apply the coupon code SAVE10	4	Correct	FR-06
R6	SAVE10 gives 10 % off	4	**Vague** (percentage not in the note)	Confirm with marketing; FLAT5 added later
R7	The system shall support card and UPI payments through a gateway	5	**Invented** (the note is a question, "ask Raj")	Out of scope for MVP until confirmed
R8	The UI shall be premium and polished	6	**Vague** (not measurable)	Replace by a test-user question (Task A, Q8)
R9	The system shall integrate with Uber Eats	6	**Invented** (the note only names a style reference)	Delete
R10	Unavailable (sold-out) items cannot be ordered	7	Correct	FR-10
R11	Checkout shall collect delivery address, name and phone	8	Correct	FR-07
R12	The system shall provide live order tracking with a map	none	**Invented**	Delete

Counts: 5 correct (R2, R4, R5, R10, R11), 4 vague (R1, R3, R6, R8), 3 invented (R7, R9, R12). Accuracy of the AI list = 5 / 12 = 41.7 % used as written; 9 / 12 = 75 % after confirming the vague ones. The corrected list is Table 1 (FR-01 to FR-10, NFR-01 to NFR-03).

L1.5 [Advanced] Ten AI test cases run against the code

The AI generated 10 test cases for cart and order placement from the requirements; they were written as Django tests (`labs_tests/test_ai10.py`) and run on the prototype **as submitted**.

ID	Test	Result	Why it failed
AI-TC-01	Add item creates cart line with quantity 1	Pass	
AI-TC-02	Add same item twice gives quantity 2	Pass	
AI-TC-03	"-" reduces quantity	Pass	
AI-TC-04	"-" on the last unit removes the line	Pass	
AI-TC-05	Cart total = sum of price x quantity ($21.97 for 2 burgers + 1 fries)	Pass	
AI-TC-06	Anonymous checkout redirects to login	Pass	(the redirect works, but the login page itself fails: DEF-02)
AI-TC-07	Empty-cart checkout redirects to the cart	Pass	
AI-TC-08	Valid order creates items and clears the cart	Pass	
AI-TC-09	Order stores name, phone and address	**Fail**	`checkout()` never reads `full_name`, `phone_number`, `address` from `request.POST` (DEF-04)
AI-TC-10	Order stores the total amount	**Fail**	`total_amount` is never set, so it stays at the default 0.00 (DEF-04)

```
prototype as submitted:  Ran 10 tests ... FAILED (failures=2)
fixed copy:              Ran 10 tests ... OK
```

8 passed, 2 failed. Both failures are real defects in the code, not wrong tests, because they come straight from FR-08 and NFR-03. The same defect is visible in the project's own database: the one order already stored in `db.sqlite3` has an empty name, empty address and total $0.00.

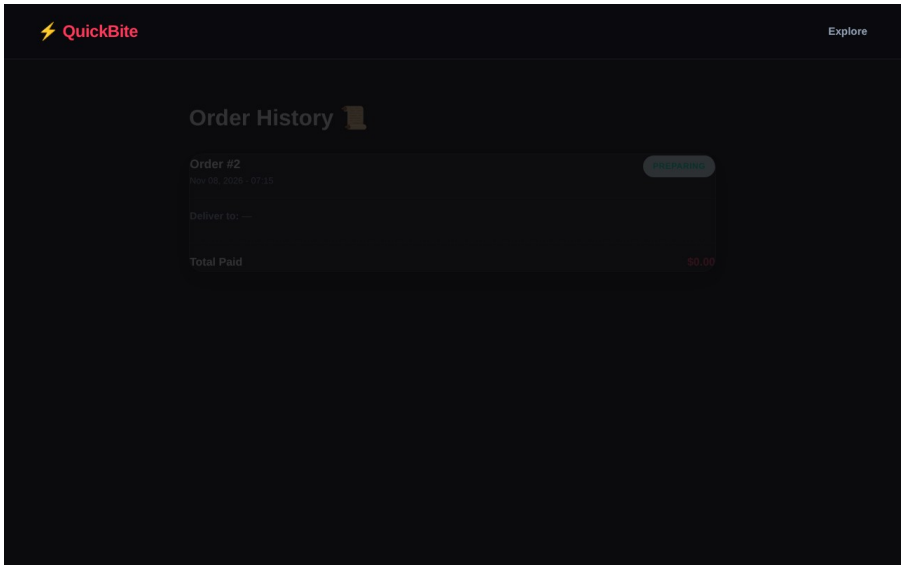


Figure 10: DEF-04 on the prototype as submitted. A real order was placed with name, phone and address filled in. Order history shows "Deliver to: -" and "Total Paid $0.00". (Food photos are placeholders, see Task E.)

Module 2 Labs: Agile, Scrum and AI-Driven Planning

L2.1 [Basic] Six practices classified under Agile Manifesto values

#	Practice in Food Delivery MVP	Manifesto value or principle
1	Show the clickable prototype to 5 users after week 2	Customer collaboration over contract negotiation
2	Keep one README and tests instead of a long specification	Working software over comprehensive documentation
3	Pull BUG-01 into the board the day it is found	Responding to change over following a plan
4	Daily 15-minute stand-up at the board	Individuals and interactions over processes and tools
5	Pair on checkout code (2 of 3 developers)	Principle: attention to technical excellence and good design
6	Release a demo every Friday	Principle: deliver working software frequently

L2.2 [Basic] Sprint goal and Definition of Done for the first iteration

Goal (first iteration, days 1 to 8): *Five test users can click through list, menu, cart, checkout and history for 10 restaurants and tell us what to change.*

Definition of Done: the screens are merged to `main` by pull request; Jenkins is green; the page loads in under 2 seconds; no Critical defect is open for the screens; every card has been tried by someone other than its author.

Three acceptance checks:

1. Given the home page, when a signed-in user opens it, then 10 restaurants are listed (FR-01).
2. Given a restaurant, when the user opens its menu, then each item shows a name and a price (FR-02).
3. Given the menu page, when the user presses “+ Add” then “Cart”, then the cart page opens in 2 taps and shows the item (FR-03, NFR-01).

L2.3 [Applied] Backlog prioritised by MoSCoW and by value versus effort

Value 1 to 5 (investor impact). Effort = points from my estimate (L2.4). Value/effort = value divided by effort.

Table A: MoSCoW

Story	Requirement	MoSCoW	Reason
US-01	Browse 10 restaurants	Must	The demo starts here
US-02	Restaurant menu	Must	Nothing to order without it
US-03	Add item to cart	Must	Core of the module
US-05	View cart and total	Must	Needed to order
US-07	Checkout details	Must	Needed to deliver
US-08	Place order and history	Must	The goal of the product
US-09	Register and sign in	Must	Checkout needs a user
US-04	Change quantity	Should	People expect it, but there is a workaround (add again)
US-10	Block unavailable items	Should	Avoids wrong orders
US-06	Coupon	Could	Marketing wants it; not required for the first demo
(not in scope)	Real payment, live tracking	Won’t (this time)	No time in 4 weeks

Table B: value versus effort

Story	Value (1 to 5)	Effort (points)	Value / effort	Rank
US-01	4	1	4.00	1
US-03	5	2	2.50	2
US-02	4	2	2.00	3 (tie)
US-05	4	2	2.00	3 (tie)
US-10	3	2	1.50	5
US-07	4	3	1.33	6
US-04	3	3	1.00	7 (tie)
US-06	3	3	1.00	7 (tie)
US-08	5	5	1.00	7 (tie)
US-09	4	5	0.80	10

Why the two tables differ. MoSCoW asks “can the product work without it?” and puts US-09 and US-08 in Must, although their value/effort is the lowest (they are expensive). Value versus effort asks “what gives the most benefit per effort?” and ranks cheap, visible stories first (US-01, US-03). I combine them: take Musts in value/effort order, so the order of work is US-01, US-03, US-02, US-05, US-07, US-09, US-08, then Should and Could.

L2.4 [Applied] Estimates: planning poker and AI

- **“Human” column:** my own solo estimates, based on the code that exists. A team round of planning poker was not held for this report, so these are one person’s cards, not a team consensus.
- **AI column:** a separate AI session received only the 10 story texts, the team size and the tech stack (not my numbers) and answered in Fibonacci points.

Story	Human (points)	AI (points)	Difference (AI minus human)	Absolute
US-01 Browse restaurants	1	2	+1	1
US-02 Menu	2	3	+1	1
US-03 Add to cart	2	5	+3	3
US-04 Change quantity	3	3	0	0
US-05 View cart and total	2	3	+1	1
US-06 Coupon	3	5	+2	2
US-07 Checkout details	3	3	0	0
US-08 Place order	5	8	+3	3
US-09 Register and sign in	5	3	-2	2
US-10 Block unavailable items	2	3	+1	1
Total	**28**	**38**	**+10**	**14**

Reasons for the biggest gaps.

- **US-03 (+3) and US-08 (+3).** The AI treats a session cart as a new “structure” (5) and order placement as the riskiest story (8: transaction, price snapshot, total recalculation). I know that the session cart is 14 lines in this project, so I gave 2; the AI over-estimated the cart. But for US-08 the AI was right about the risk: the prototype’s order code lost the delivery details and total (DEF-04) and had no transaction (L5.3 R1). My 5 was too low for the risk, even if not for the typing effort. Lesson: effort and risk are different things; the AI mixed them in the cart and was right in the order.
- **US-09 (-2).** The AI assumed Django’s built-in authentication makes sign-in 3 points. In the prototype the login template path pointed to a file that does not exist and no register page was wired, so the home

page and the sign-in page crash (DEF-01, DEF-02): my 5 reflects that wiring and templates still need work.

- **US-06 (+2).** The AI included rounding and floor-at-zero cases; they are real (EC-C6), and I would raise my 3 to 5.

L2.5 [Advanced] Three benefits and three challenges of AI in the Agile work

#	Benefit	Evidence in this report	Challenge	Evidence in this report
1	Fast drafting of stories, questions and test cases	10 AI test cases found 2 real defects (L1.5)	Invented content	3 of 12 AI requirements invented (L1.4); 4 of 12 AI tests wrong (L4.4)
2	A second opinion on estimates and risk	AI gave 8 to US-08; defects were later found there (L2.4, L5.3)	Over-estimation and false certainty	AI gave 5 points to a 14-line cart (L2.4); one wrong number in the bottleneck analysis (Task B)
3	Quick analysis of board data	AI named Design as the bottleneck and the main numbers were right (Task B)	Needs human checking; time cost of checking	2 of 8 review suggestions were false (L5.3); every suggestion needed a test

Module 3 Labs: Prompting, Git, Jenkins and Docker

L3.1 [Basic] Four versions of a prompt for add-to-cart, scored 1 to 5

Each version was sent to a fresh AI session (Claude Haiku 5.5), so earlier answers could not influence later ones. **Scoring rubric:** score = 1 + points for each of the following, worth 1 if fully met, 0.5 if partly met, 0 if not: (a) fits the project (model `FoodItem`, cart format `{str(id): int}`, works with the existing links), (b) behaves as required (404 for unknown or unavailable item, maximum 20), (c) secure (state change only on POST, redirect checked), (d) tests supplied and runnable.

Ver.	Prompt (shortened)	What came back	a	b	c	d	Score
1	“write add to cart in django”	A generic `Cart` class for a `Product` model with CART_SESSION_ID in settings, **price stored in the session**, a quantity from user input, a template form. Not usable in the project	0	0	0	0	**1**
2	“Write a Django view add_to_cart(request, item_id) for a food delivery website. The cart must be stored in the session.”	One view on a `MenuItem` model, 404 if unavailable, `@require_POST`, quantity from user input (no upper limit), **unchecked `Referer` redirect**	0	1	0.5	0	**2.5**
3	Version 2 plus: Django 6; `FoodItem(name, price, is_available)`; cart `{str(id): int}`; 404 if missing or unavailable; maximum 20; redirect back or to `restaurant_list`; `session.modified`; “return only the code”	20-line view: `get_object_or_404`, `min(qty + 1, 20)`, redirect only if `url_has_allowed_host_and_scheme` passes. Still changes the cart on GET, and crashes on a legacy `{'quantity': n}` value	1	1	0.5	0	**3.5**
4	Few-shot: my `remove_from_cart` as the style example; same constraints; `@require_POST`; safe redirect; “then write 3 unit tests”	View in the project style with `@require_POST`, cap 20, safe redirect, plus 3 tests	0.5*	1	1	0.5**	**4**

* POST-only breaks the existing “+ Add” links, so the templates need small forms with `{% csrf_token %}`. ** The AI’s own tests failed on the first run with `IntegrityError: NOT NULL constraint failed: restaurants_fooditem.restaurant_id` because the fixture did not create a restaurant, although the AI

wrote “adjust the fields to match the model”. I give half a point for the tests because the code is right and the fix to the test is one line. The half point in (a) is because the POST-only view needs a small template change.

Version 4 prompt (few-shot) and output (code only)

```
# prompt example given to the AI (the project's own style)
def remove_from_cart(request, item_id):
    cart = request.session.get('cart', {})
    key = str(item_id)
    if key in cart:
        if cart[key] > 1:
            cart[key] -= 1
        else:
            del cart[key]
    request.session['cart'] = cart
    request.session.modified = True
    return redirect('view_cart')

# AI output for add_to_cart
@require_POST
def add_to_cart(request, item_id):
    item = get_object_or_404(FoodItem, pk=item_id, is_available=True)
    cart = request.session.get('cart', {})
    key = str(item_id)
    cart[key] = min(cart.get(key, 0) + 1, MAX_QTY)
    request.session['cart'] = cart
    request.session.modified = True
    referer = request.META.get('HTTP_REFERER', '')
    if referer and url_has_allowed_host_and_scheme(
        referer, allowed_hosts={request.get_host()},
        require_https=request.is_secure()):
        return redirect(referer)
    return redirect('restaurant_list')
```

I also ran the version-3 and version-4 code inside a copy of the project against my 7 behaviour tests: version 3 passed 5 of 7 (it failed on the legacy dict value, and on “unknown id is stored”, which is actually the better behaviour); version 4 failed all 7 because it is POST-only and my tests use GET links, which is correct for security but needs the template change. **Finding:** each better prompt gave a better answer (scores 1, 2.5, 3.5, 4); the biggest jump came from naming the real model and the cart format (version 2 to 3), and the security quality came from stating security rules explicitly and giving an example in the project’s style (version 4).

L3.2 [Applied] AI refactors one function; tests prove the behaviour is unchanged

Function: `add_to_cart`. The quantity-reading expression appeared 4 times in `views.py`; the AI was asked to remove the nested `if` and the repeated expression, without changing behaviour.

```
# BEFORE (14 lines)
def add_to_cart(request, item_id):
    cart = request.session.get('cart', {})
    str_item_id = str(item_id)

    current_qty = 0
    if str_item_id in cart:
        val = cart[str_item_id]
        current_qty = val.get('quantity', 0) if isinstance(val, dict) else int(val)

    cart[str_item_id] = current_qty + 1
    request.session['cart'] = cart
    request.session.modified = True
    return redirect(request.META.get('HTTP_REFERER', 'restaurant_list'))
```

```
# AFTER (12 lines)
def _quantity(value):
    """Cart values may be a dict (old format) or an int."""
    return value.get('quantity', 0) if isinstance(value, dict) else int(value)

def add_to_cart(request, item_id):
    cart = request.session.get('cart', {})
    key = str(item_id)
    cart[key] = _quantity(cart.get(key, 0)) + 1
    request.session['cart'] = cart
    request.session.modified = True
    return redirect(request.META.get('HTTP_REFERER', 'restaurant_list'))
```

Tests. Seven *characterization* tests were written before the refactor (`test_refactor_add_to_cart.py`): new item starts at 1; repeated adds increment; a legacy `{'quantity': 4}` value is upgraded to 5; other items are untouched; redirect goes back to the referer; redirect goes to the list when there is no referer; an unknown id keeps today's behaviour (it is stored; this is the defect DEF-05 and is **kept on purpose** so that the refactor changes nothing).

```
BEFORE refactor : Ran 7 tests ... OK
AFTER  refactor : Ran 7 tests ... OK
MUTATION CHECK  : change "+ 1" to "+ 2" in the refactored code -> Ran 7 tests ... FAILED
(failures=5)
```

The mutation check shows that the 7 tests would have caught a behaviour change, so "7 passed" is meaningful evidence.

L3.3 [Basic] A merge conflict in cart and order placement

Two branches changed the same line of `cart.html`, the checkout button label, in a copy of the repository.

```
$ git checkout -b feature/cart-checkout-label      # developer 1
df305ef Cart: friendlier checkout button label
$ git checkout main && git checkout -b feature/coupon-ui   # developer 2
f655287 Cart: hint about coupon on checkout button
$ git checkout main && git merge feature/cart-checkout-label
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git merge feature/coupon-ui
Auto-merging orders/templates/orders/cart.html
CONFLICT (content): Merge conflict in orders/templates/orders/cart.html
Automatic merge failed; fix conflicts and then commit the result.
$ git status --short
UU orders/templates/orders/cart.html
```

Conflict markers in the file:

```
<<<<<<< HEAD
            <a href="{% url 'checkout' %}" class="checkout-btn">Checkout securely →</a>
=======
            <a href="{% url 'checkout' %}" class="checkout-btn">Pay now (apply coupon
first)</a>
>>>>>>> feature/coupon-ui
```

What the AI suggested: keep one label, because a button cannot have two, and do not lose the coupon hint: move the hint into a tooltip (`title`) attribute. **My resolution:** I accepted this because it keeps both intents (a friendlier label and a coupon reminder) and it does not make the button text longer on a phone.

```
            <a href="{% url 'checkout' %}" class="checkout-btn" title="Apply your coupon
first">Checkout securely →</a>
$ git add orders/templates/orders/cart.html && git commit -m "Merge feature/coupon-ui:
resolve checkout button conflict"
*   bf2110c Merge feature/coupon-ui: resolve checkout button conflict
|\
| * f655287 Cart: hint about coupon on checkout button
* | df305ef Cart: friendlier checkout button label
|/
* 4fd4813 Initial cart page (from prototype)
```

L3.4 [Applied] Jenkinsfile with checkout, build, test and package; one failed and one successful build

The `Jenkinsfile` (full listing in Appendix C) has the four stages, `triggers { githubPush() }` and a `post { failure { mail ... } }` block. The Test stage runs 35 tests.

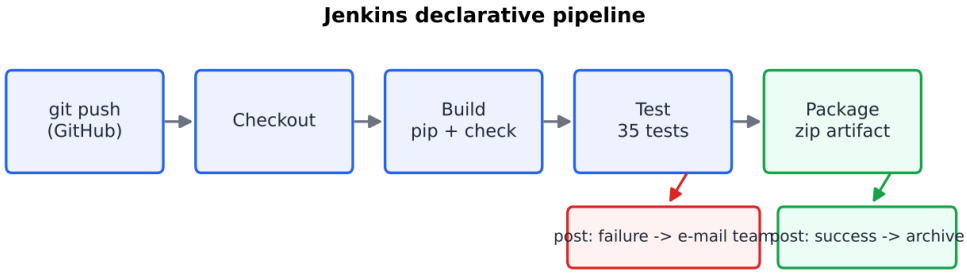


Figure 11: The CI/CD pipeline of the project.

> **Evidence note.** No Jenkins server was available. The logs below come from `labs/run_stages.sh`, which runs the *same shell commands* as each stage on the real code and prints them in Jenkins style. The e-mail step is simulated because the sandbox has no mail server. No Jenkins screenshots are included because no server was run.

Build #1: the prototype as submitted (FAILED).

```
Started by GitHub push (emulated)  - Build #1
[Pipeline] { (Checkout) }       > git checkout main
[Pipeline] { (Build) }          + pip install -r requirements.txt       requirements
resolved
                                + python manage.py check                System check
identified no issues
                                + python manage.py makemigrations --check --dry-run   No
changes detected
[Pipeline] { (Test) }           + python manage.py test labs_tests.test_ai10 ...
   ERROR: test_IT_07_home_page_loads_for_anonymous_visitor
   ERROR: test_IT_08_login_page_loads
   FAIL:  test_AI_TC_09_order_stores_delivery_details
   FAIL:  test_AI_TC_10_order_stores_total_amount
   FAIL:  test_IT_02_full_order_flow_and_history
   FAIL:  test_IT_04_checkout_page_shows_total
   FAIL:  test_IT_05_cannot_add_unavailable_item
   FAIL:  test_IT_06_cannot_add_nonexistent_item
   FAIL:  test_IT_09_empty_delivery_details_rejected
   (and 8 more: coupon tests EC-C1/C2/C3/C6/C7, EC-Q4, UT-01, UT-04: features not built
yet)
   Ran 35 tests in 10.273s    FAILED (failures=13, errors=4)
ERROR: script returned exit code 1
Stage 'Package' skipped due to earlier failure(s)
[Pipeline] mail  -> to: team@example.com | subject: FAILED: FoodDeliveryMVP #1
(simulated)
Finished: FAILURE
```

Fix. Of the 17 failures, 9 are real defects (DEF-01 to DEF-06) and 8 test features that were not built (coupon FR-06, quantity cap, helper functions). The fix: register and login wiring (DEF-01, DEF-02), checkout total variable (DEF-03), save delivery details and total (DEF-04), reject unavailable or unknown items (DEF-05), server-side validation (DEF-06), plus the coupon feature and the quantity cap.

Build #2: the fixed copy (SUCCESS).

```
Started by GitHub push (emulated)  - Build #2
[Pipeline] { (Test) }           + python manage.py test labs_tests.test_ai10
labs_tests.test_cart_order labs_tests.test_equivalence
   Ran 35 tests in 10.298s   OK
[Pipeline] { (Package) }        + zip -qr foodmvp-2.zip config orders restaurants static
manage.py requirements.txt Dockerfile docker-compose.yml
   archiveArtifacts: foodmvp-2.zip (52 KB)
Build, tests and package passed - artifact archived.
Finished: SUCCESS
```

L3.5 [Advanced] Cart and order placement with a database in Docker Compose

`docker-compose.yml` (listing in Appendix C) starts two services: `db` (PostgreSQL 16 with a volume and a health check) and `web` (the Django app, which waits until `db` is healthy, then runs `migrate`, `seed_restaurants` and the server). The settings read `POSTGRES_HOST`, `POSTGRES_USER` and `POSTGRES_PASSWORD` from the environment, so no password is stored in the repository; the compose file refuses to start if `POSTGRES_PASSWORD` is not set.

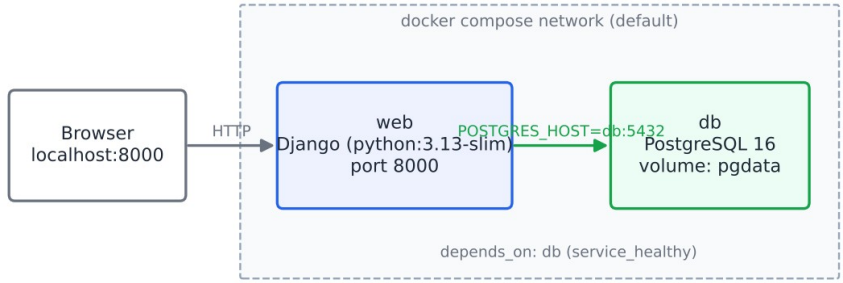


Figure 12: How the two containers communicate.

Check	Result
`POSTGRES_PASSWORD=x docker compose config`	Valid: both services and the `pgdata` volume are listed
`docker compose config` without the password	Stops with "required variable POSTGRES_PASSWORD is missing a value" (the intended safety check)
Django settings with `POSTGRES_HOST=db`	Uses `django.db.backends.postgresql`, host `db`
`python manage.py check`	No issues

Real run (Docker Engine 29.9.2, `docker compose up`, PostgreSQL 16). I started the project's own `Dockerfile` and `docker-compose.yml` unchanged. The sandbox cannot reach Docker Hub, so I could not pull the official `python:3.13-slim` and `postgres:16-alpine` images. I built **substitute base images with the same tags from Ubuntu 24.04 packages** (Python 3.12.3 and PostgreSQL 16.2) and ran the compose file on top of them. An extra override file (not part of the project) only gave the image build access to the package proxy. So this proves that the Dockerfile builds, that the two containers start in the right order, and that the app and the database work together. It does not test the official images, and the app ran on Python 3.12, not 3.13.

```
$ docker compose ps
NAME          IMAGE                COMMAND                  SERVICE   CREATED          STATUS                    PORTS
fixed-db-1    postgres:16-alpine   "docker-entrypoint.s…"   db        About a minute   Up About a minute (healthy)   5432/tcp
fixed-web-1   fixed-web            "sh -c 'python manag…"   web       About a minute   Up About a minute             0.0.0.0:8000->8000/tcp

db-1   | database system is ready to accept connections
web-1  | Applying orders.0001_initial... OK  (and 20 more migrations)
web-1  | Successfully seeded 10 restaurants
web-1  | System check identified no issues (0 silenced).
web-1  | Starting WSGI development server at http://0.0.0.0:8000/
web-1  | "POST /orders/register/ 302", "GET /orders/cart/add/1/ 302" (x2), "POST
/orders/checkout/ 302"

$ docker compose exec db psql -U foodmvp -c "select id, full_name, phone_number,
total_amount, status from orders_order;"
 id |  full_name  |  phone_number   | total_amount |  status
----+-------------+-----------------+--------------+-----------
  1 | Docker Demo | +91 90000 00000 |        17.98 | Preparing
```

I drove the browser (Chromium) through register, two "+ Add" clicks, the cart, checkout and history. The order is stored in PostgreSQL with the name, phone and total (the prototype stored none of them, DEF-04), and the 10 seed restaurants are in the `db` container. The `db` service has a health check, and `web` waits for it (`depends_on: condition: service_healthy`), so the migration did not start before the database was ready. The phone

number and name are test data. The database password was generated for the run and is not stored in the repository. Figures 12a to 12c show the run.

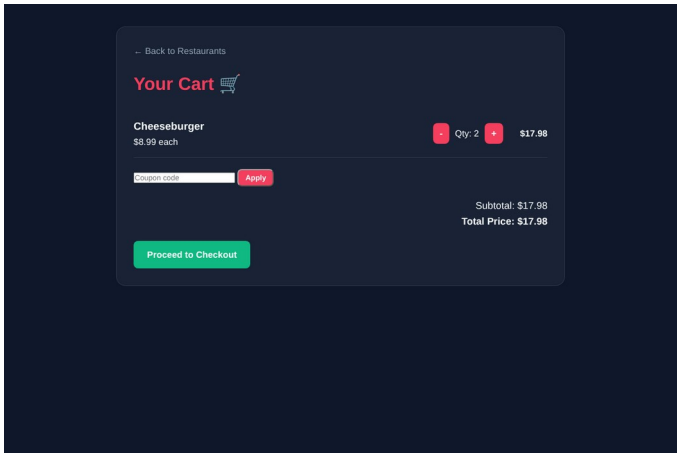


Figure 12a: The cart inside the container.

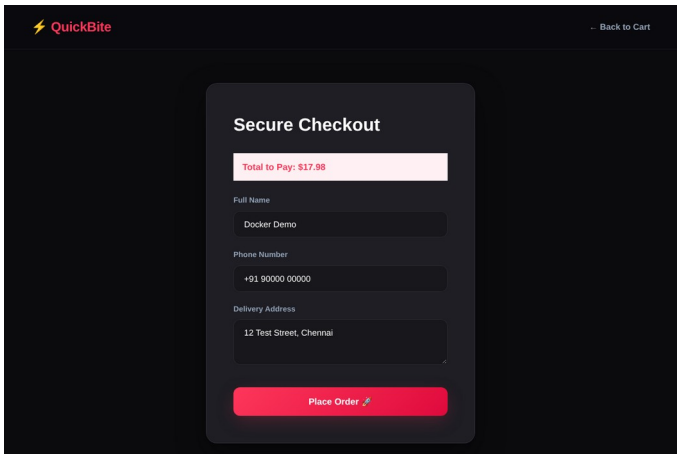


Figure 12b: Checkout with delivery details.

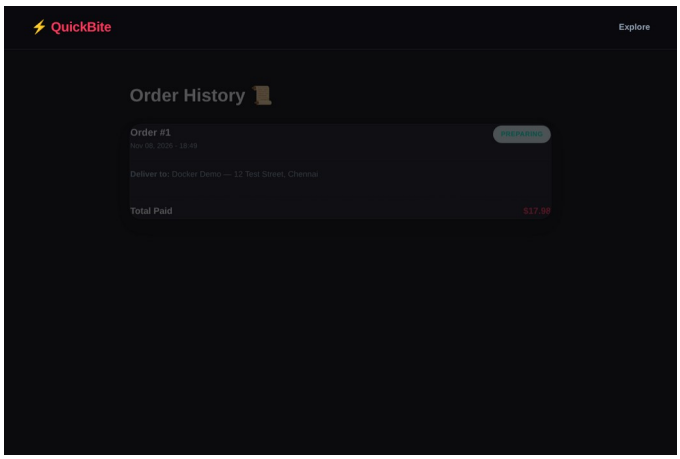


Figure 12c: Order history read back from PostgreSQL ($17.98).

Module 4 Labs: Testing, Selenium and Monitoring

L4.1 [Basic] Ten manual test cases, executed, and two defects

Executed on the prototype **as submitted** (steps done in a browser session driven by a script for repeatability; results match the automated tests). Test data: restaurant "Burger Joint", items Cheeseburger $8.99 and Crispy Fries $3.99; user `alice`.

ID	Requirement	Steps	Expected	Actual	Result
TC-01	FR-03	Open the menu; press "+ Add" on Cheeseburger	Cart has Cheeseburger x 1	Cart has Cheeseburger x 1	Pass
TC-02	FR-04	Press "+ Add" on the same item twice	Quantity 2	Quantity 2	Pass
TC-03	FR-04	In the cart, press "-" at quantity 2	Quantity 1	Quantity 1	Pass
TC-04	FR-04	Press "-" at quantity 1	Line removed	Line removed	Pass
TC-05	FR-05	Add 2 burgers and 1 fries; open the cart	Total $21.97	Total $21.97	Pass
TC-06	FR-05	Open the cart with nothing in it	"Cart is empty" message	Message shown	Pass
TC-07	FR-10	As a visitor, open checkout	Redirected to a working sign-in page	Redirect happens, then the **sign-in page returns an error 500**	**Fail** (DEF-02)
TC-08	FR-07	Signed in, empty cart, open checkout	Redirected to the cart	Redirected to the cart	Pass
TC-09	FR-07	Signed in, 1 burger in the cart, open checkout	"Total to Pay: $8.99"	"Total to Pay: $" (blank)	**Fail** (DEF-03)
TC-10	FR-08, NFR-03	Fill name, phone, address; press "Place Order"	Order saved with those values and total $8.99	Order saved with empty name/phone/address and total $0.00; history shows "Deliver to: -"	**Fail** (DEF-04)

Result: 7 passed, 3 failed. Defects found while testing (all are reported with steps, expected result, actual result and severity):

ID	Title	Steps to reproduce	Expected	Actual	Severity
DEF-04	Order loses delivery details and total	1 Sign in. 2 Add an item. 3 Checkout: enter "Alice Rao", "+91 98765 43210", "12 MG Road, Pune". 4 Place Order. 5 Open order history	Order shows the name, address and total	`Deliver to: -, Total Paid $0.00`. Cause: `checkout()` does not read the form fields or compute `total_amount`	**Critical** (a real order cannot be delivered or paid)
DEF-03	Checkout total is blank	1 Add an item. 2 Open `/orders/checkout/`	"Total to Pay: $8.99"	"Total to Pay: $". Cause: the view passes `total_price` but the template prints `total`	**Major**

Further defects from the same session and the automated runs:

ID	Title	Severity	Evidence
DEF-01	Home page returns error 500 for a visitor who is not signed in (`NoReverseMatch: 'register'`; the link exists in the template but no `register` URL exists)	Critical	Figure 13, IT-07
DEF-02	`/accounts/login/` returns error 500 (`TemplateDoesNotExist: restaurants/login.html`; the file is in `orders/templates/orders/`)	Critical	Figure 14, IT-08

ID	Title	Severity	Evidence
DEF-05	Unavailable or non-existent items can be added to the cart (`/orders/cart/add/99999/` stores id 99999)	Medium	IT-05, IT-06
DEF-06	Delivery fields are validated only by the browser (`required`); the server accepts an empty order	Medium	IT-09
DEF-07	Cart changes happen on GET links, and the redirect after adding trusts the `Referer` header (open redirect)	Medium	R3a, R3b in L5.3
UX-01	No feedback after “+ Add”; the cart link shows no item count	Low	Task D session

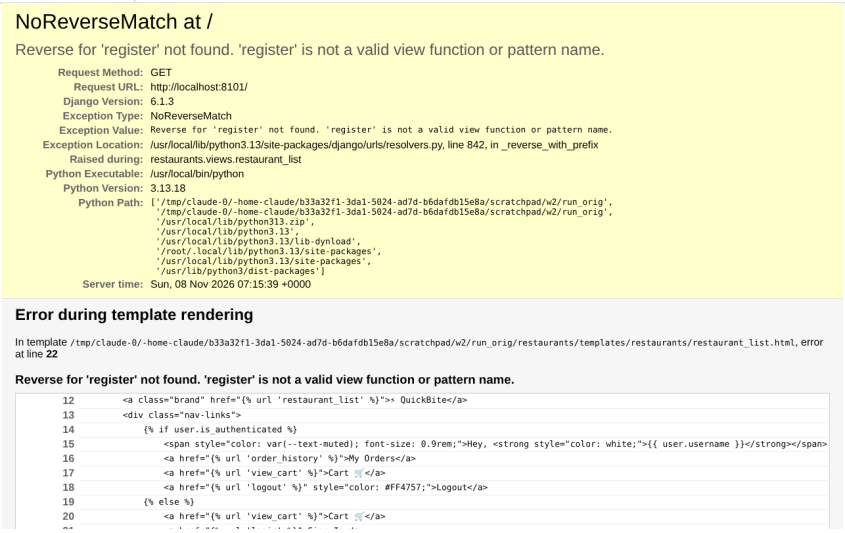


Figure 13: DEF-01. The home page for a visitor who is not signed in.

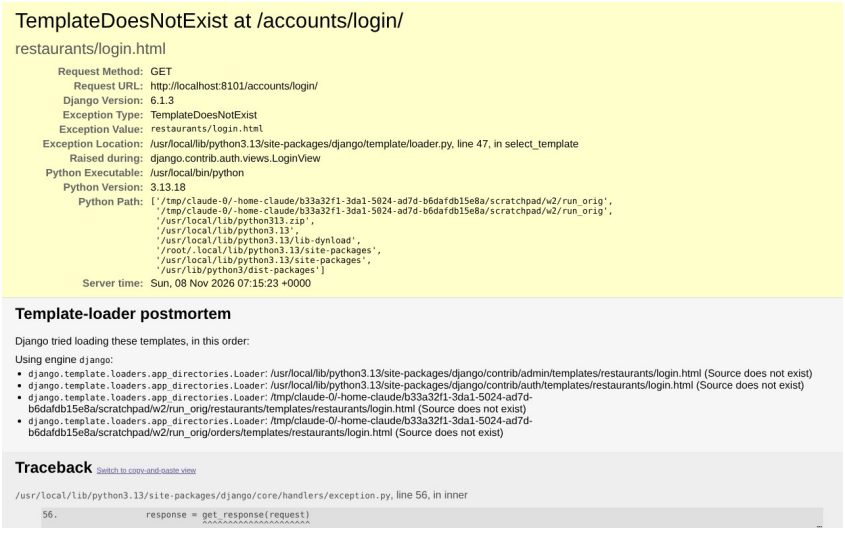


Figure 14: DEF-02. The sign-in page. The error page is the Django debug page because DEBUG is True.

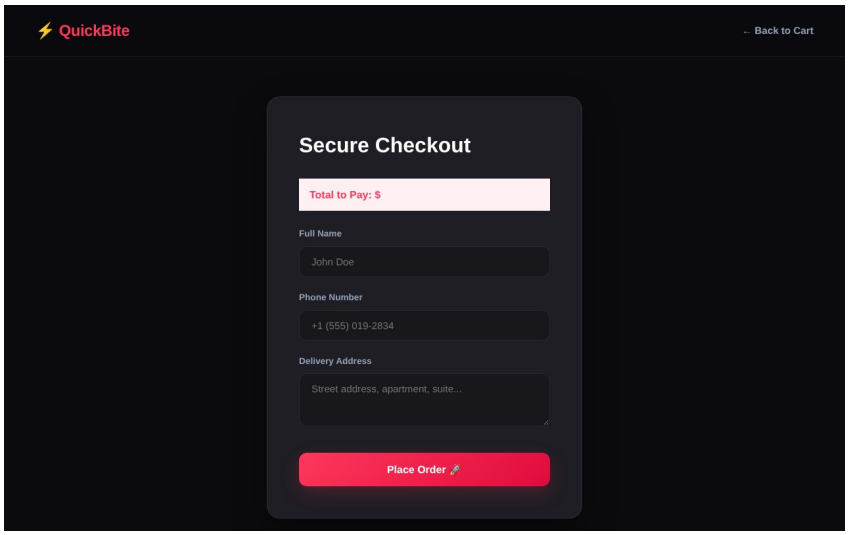


Figure 15: DEF-03. The checkout page shows "Total to Pay: $" with no number.

L4.2 [Basic] Five acceptance scenarios (Given-When-Then) and their execution

ID	Scenario	Prototype as submitted	Fixed copy
AS-1	**Given** a signed-in customer with 3 items in the cart, **when** she applies SAVE10 and places the order, **then** the order total is $13.03 and the cart is empty	Fail (no coupon box)	Pass (Selenium S1)
AS-2	**Given** a signed-in customer at checkout, **when** she leaves the address empty and submits, **then** no order is created and an error is shown	Fail (an empty order is created, DEF-06)	Pass (IT-09)
AS-3	**Given** a visitor who is not signed in, **when** she opens the checkout, **then** she is taken to a working sign-in page	Fail (error 500, DEF-02)	Pass (AI-TC-06 + IT-08)
AS-4	**Given** a sold-out item, **when** a customer tries to add it, **then** it is not added	Fail (it is added, DEF-05)	Pass (IT-05)
AS-5	**Given** a placed order, **when** the customer opens order history, **then** it shows her name, address and total	Fail (blank details, DEF-04)	Pass (IT-02)

> **Evidence note.** The five scenarios were executed by automated tests (the Result column above), not by a classmate acting as the client. No classmate sign-off is claimed. In a real project the product owner would run AS-1 to AS-5 on the fixed copy and mark each Accepted or Rejected before release.

L4.3 [Applied] The Selenium script converted to the Page Object Model

Both versions run in headless Chromium 153 with 3 scenarios each (S1 SAVE10 total $13.03, S2 FLAT5 total $9.48, S3 invalid code total $14.48); both pass (`Ran 3 tests ... OK`).

Version 1: plain script (one test shown).

```
def test_S1_save10(self):
    d = self.driver
    self.login_by_cookie()
    for i in range(3):
        d.get(f'{self.live_server_url}/restaurant/{self.rest.id}/')
        d.find_elements(By.CSS_SELECTOR, 'a.btn-card')[i].click()
    d.get(f'{self.live_server_url}/orders/cart/')
    d.find_element(By.ID, 'coupon').send_keys('SAVE10')
    d.find_element(By.ID, 'apply-coupon').click()
    WebDriverWait(d, 5).until(EC.presence_of_element_located((By.ID, 'subtotal')))
    self.assertEqual(d.find_element(By.ID, 'total').text, '13.03')
    d.find_element(By.ID, 'checkout-link').click()
    d.find_element(By.NAME, 'full_name').send_keys('Alice Rao')
    d.find_element(By.NAME, 'phone_number').send_keys('+91 98765 43210')
    d.find_element(By.NAME, 'address').send_keys('12 MG Road, Pune')
```

```
    d.find_element(By.CSS_SELECTOR, 'button.btn-submit').click()
    self.assertIn('/orders/history/', d.current_url)
```

Version 2: Page Object Model (page classes in Appendix C).

```
def test_S1_save10(self):
    self.login_by_cookie()
    MenuPage(self.driver, self.live_server_url).add_first_n_items(self.rest.id, 3)
    cart = CartPage(self.driver, self.live_server_url).open().apply_coupon('SAVE10')
    self.assertEqual(cart.total(), '13.03')
    cart.proceed_to_checkout().place_order('Alice Rao', '+91 98765 43210', '12 MG Road,
Pune')
    self.assertIn('/orders/history/', self.driver.current_url)
```

Maintenance comparison (measured)

Measure	Plain script (3 tests)	POM (3 tests + `pages.py`)
Non-blank lines of code	74	29 + 39 = 68
Locators written inline	33	10 (all in `pages.py`)
Places that mention the “Apply” button	3	1
Places that mention the “Checkout” link	3	1
Test readability	Technical steps	Reads like the story

Note. If the designer renames `#apply-coupon`, the plain script needs 3 edits (one per test) and the POM needs 1. With one test the effort is the same, so POM pays off from the second test. The cost of POM is an extra file and an extra layer to learn; for a 4-week prototype I would start with POM once there is more than one scenario.

L4.4 [Applied] AI-generated tests: keep, remove, add

The AI produced 12 raw tests for the cart and order module (`test_ai_raw_L44.py`). Validity was judged against the requirements, not against what the code happens to do.

ID	Raw AI test	Valid?	Reason	Run on submitted code
A01	Add returns a redirect	Valid	FR-03	Pass
A02	Quantity increments	Valid	FR-04	Pass
A03	“-” never goes negative	Valid	FR-04	Pass
A04	Cart total correct ($10.49)	Valid	FR-05	Pass
A05	Anonymous checkout is blocked	Valid	FR-10	Pass
A06	Order saves items and clears cart	Valid	FR-08	Pass
A07	Order saves the delivery address	Valid	FR-07, NFR-03	**Fail (DEF-04)**
A08	Order history lists orders	Valid	FR-09	Pass
A09	Add-to-cart must be POST-only (405 on GET)	**Wrong**	Not a requirement of the MVP (it is a security improvement, DEF-07, not a defect of the stated requirement)	Fail
A10	Cart adds 5 % tax	**Wrong**	No tax in the scenario or the notes	Fail
A11	“out of stock” message on a sold-out item	**Wrong**	The model has `is_available` only; no stock or message requirement	Error
A12	Cart endpoint returns JSON	**Wrong**	The app renders HTML pages	Fail

Accuracy = 8 valid / 12 generated = 66.7 %. Of the 4 wrong tests, A09 is the one I turned into a real backlog item (DEF-07); the other 3 were removed.

Missing tests added by me (the AI did not think of them): empty cart (TC-06), unavailable item cannot be added (IT-05), unknown id (IT-06), empty delivery details (IT-09), checkout shows the total (IT-04), history shows only the user's own orders (IT-03), quantity cap 20 (EC-Q4), coupon classes (EC-C1 to C7), and a corrupt or negative quantity (R2). **Final set:** the 8 valid AI tests were kept (they are covered by `test_ai10` and `test_cart_order`), the 4 wrong ones were removed, and the missing ones were added; the final suite is the 35 tests of the Jenkins build (`test_ai10` 10, `test_cart_order` 13, `test_equivalence` 12). Final run on the fixed copy: `Ran 35 tests ... OK`; on the submitted code: 17 fail (13 failures, 4 errors).

L4.5 [Advanced] Monitoring dashboard with 6 metrics, thresholds and alert actions

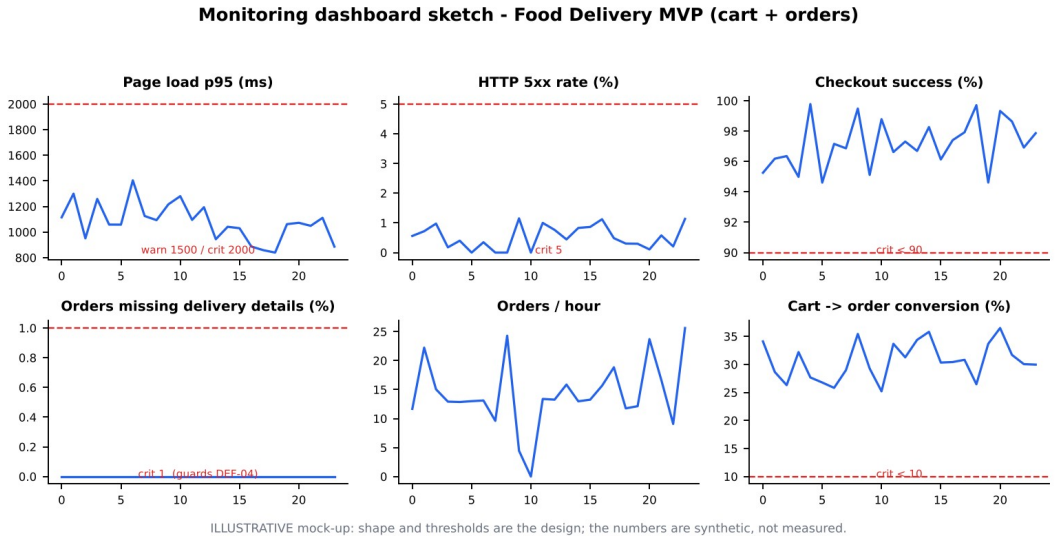


Figure 16: Dashboard sketch. The numbers in the sketch are synthetic; the six panels and the thresholds are the design.

#	Metric	How collected	Warn	Critical	Alert action
1	Page load, 95th percentile (ms)	Probe of menu and cart pages every minute	1500	2000 (NFR-02)	Check image sizes and database queries; roll back the last release if it started after a deploy
2	HTTP 5xx error rate (%)	Probe and server log	1	5	Page the developer on duty; open a defect (this would have caught DEF-01 and DEF-02 at once)
3	Checkout success rate (%)	Server log: completed orders divided by started checkouts	below 95	below 90	Stop the current release; check Jenkins and the last merge
4	Orders missing details or total (%)	Database query on `Order`	0.1	1	Block new deploys; fix immediately (guards DEF-04, NFR-03)
5	Orders per hour	Database count	falls 50 % below the same hour last week	0 orders for 1 hour at peak	Check that the site is reachable; tell the product owner
6	Cart-to-order conversion (%)	Needs analytics events	below 20	below 10	Product review: look at the checkout page and the coupon step

`labs/monitoring/monitor.py` implements metrics 1, 2, 4 (and the zero-total check) against the real app. **Real output**, run against a copy of the project database, which already holds one order placed through the app, plus one order placed by the script:

```
PROTOTYPE AS SUBMITTED                  FIXED COPY
page_load_p95_ms            3.1  OK     page_load_p95_ms            3.8  OK
```

```
http_5xx_rate_pct           0.0  OK     http_5xx_rate_pct           0.0  OK
orders_missing_details_pct  100  CRIT   orders_missing_details_pct   50  CRIT
orders_zero_total_pct       100  CRIT   orders_zero_total_pct        50  CRIT
```

The monitor catches the data defect (DEF-04) that the speed and error metrics cannot see. On the fixed copy 50 % is still critical, correctly, because the old order from before the fix remains in the database; it needs a data repair. Metrics 3, 5 and 6 need request logs or analytics that the prototype does not have.

Module 5 Labs: SQA, SonarQube and Ethical AI

L5.1 [Basic] Review with a checklist and the defects found

> **Evidence note.** The sheet asks for a classmate's code or requirement list. No classmate's work was available, so the checklist was applied to this project's own cart and order module and its requirement list (Table 1). No classmate review is claimed.

#	Checklist question	Met?	Finding
1	Does every requirement have an ID and a test?	No	Before this report there was no test at all (`orders/tests.py` is empty)
2	Does every form field that the user types get stored?	**No**	`full_name`, `phone_number`, `address` ignored (DEF-04)
3	Are totals computed and shown correctly?	**No**	Blank total at checkout (DEF-03); `total_amount` never set (DEF-04)
4	Do all pages open for every kind of user?	**No**	Home page for visitors (DEF-01); sign-in page (DEF-02)
5	Is user input validated on the server?	**No**	Empty delivery details accepted (DEF-06); unknown and sold-out items accepted (DEF-05)
6	Do state-changing actions use POST and CSRF?	**No**	Cart changes by GET link; unchecked `Referer` redirect (DEF-07)
7	Are secrets kept out of the code?	**No**	`SECRET_KEY` is in `settings.py`
8	Is debug mode off and are allowed hosts set for release?	**No**	`DEBUG = True`, `ALLOWED_HOSTS = []`
9	Is duplicated code removed?	Partly	Quantity expression copied 4 times
10	Are unused files and imports removed?	No	Unused `success.html`; unused import `get_object_or_404` in `orders/views.py`
11	Is the data model consistent with the migrations?	Yes	`makemigrations --check` reports no changes
12	Are names clear and consistent with the requirements?	Yes	`view_cart`, `order_history`, `FoodItem` are clear
13	Does the user get feedback after an action?	No	No message or cart count after "+ Add" (UX-01)
14	Is personal data handled carefully?	Partly	Order stores phone and address (needed); no real data in the repository; the database file `db.sqlite3` is committed

Result: of 14 questions, 10 were not met, 2 were partly met and 2 were met; 8 distinct defects and issues are reported (DEF-01 to DEF-07, UX-01) plus 3 hygiene findings (items 7, 8, 10).

L5.2 [Applied] Three code-quality findings in detail (rule, risk, fix)

> **Evidence note.** SonarQube was not available, so these findings come from bandit, pylint and Django's `check --deploy`, which report the same kind of issues. In SonarQube they would appear as one vulnerability or hotspot, one code smell and one duplication issue.

	Finding 1: hard-coded secret	Finding 2: duplicated quantity logic	Finding 3: debug mode and open hosts
Tool and rule	bandit B105 (`hardcoded_password_string`), Low severity, `config/settings.py:23`	Duplication check: the expression `val.get('quantity', 0) if isinstance(val, dict) else int(val)` appears 4 times in `orders/views.py`; duplication 4.9 % (4-line window)	Django `check --deploy`: security.W018 (`DEBUG` is True) and security.W020 (`ALLOWED_HOSTS` is empty)
Risk	Anyone who can read the repository knows the key that signs sessions and cookies; they could forge a session	A fix in one copy and not the others gives different cart behaviour in different pages (the corrupt-value crash in L5.3 R2 would have to be fixed 4 times)	With debug on, an error page shows source code, settings and variables to the visitor, as Figure 14 shows for DEF-02
Fix	Read the key from the environment: `SECRET_KEY = os.environ['DJANGO_SECRET_KEY']`; create a new key and never commit it	One helper `_qty()`; all 4 copies call it (done in the fixed copy; duplication 0 %, pylint 7.56 to 8.76)	`DEBUG = os.environ.get('DEBUG') == '1'`; `ALLOWED_HOSTS` from the environment (done for hosts in the fixed copy); set both in the Docker Compose file

L5.3 [Applied] AI code review of a pull request; every suggestion tested before acceptance

The pull request is the change from the prototype to the fixed copy (`orders/views.py`, 222 lines of diff: checkout saves details and total, coupon, helper functions, register view). The AI (a separate session that saw only the diff) returned 8 suggestions; suggestion 3 has two parts, so the table has 9 rows (R1 to R8, with R3a and R3b). For each I wrote a test that asserts the desired behaviour (`test_ai_review_checks.py`) and ran it on the code before the review.

ID	AI suggestion	Test result before change	Verdict	Decision
R1	Create the order and its items inside `transaction.atomic()`	**Fail**: when the 2nd item fails, an orphan `Order` stays (1 order instead of 0)	Real (High)	**Accepted**: `with transaction.atomic()` added
R2	A corrupt or negative quantity in the session must not crash or reduce the total	**Fail**: `'abc'` raised `ValueError`; quantity -3 gave total $-26.97	Real (High)	**Accepted**: `_qty` clamps to 0..20 and skips bad lines
R3a	Cart actions must not change state on GET; use `@require_POST` and CSRF forms	**Fail**: GET returns 302 and changes the cart	Real (High)	**Deferred**: the “+ Add” and “-” buttons are links; needs template forms (BUG-03)
R3b	Do not redirect to an unvalidated `Referer`	**Fail**: redirect went to `https://evil.example/phish`	Real (High)	**Accepted**: `url_has_allowed_host_and_scheme` added
R4	A coupon can make the total $0.00 and be reused without limit; decide the rules	**Fail**: a $0.00 order is accepted	Real (Medium), but a business decision	**Deferred** to the product owner (BUG-04)
R5	Keep the user’s input when the form shows an error; use a Django Form	not tested (style)	Reasonable (Medium)	**Deferred** (UX improvement)
R6	Use `django.contrib.messages` instead of a session key for the coupon error	not tested (style)	Low value (the reviewer itself admitted no bug)	**Rejected** for this release
R7	`register` may raise `ValueError` because `login()` has no backend	**Pass**: with one authentication backend (our configuration) registration works	**False positive**	**Rejected**

ID	AI suggestion	Test result before change	Verdict	Decision
R8	`order_history` has an N+1 query problem	**Pass**: exactly 3 queries for 3 orders (the template does not read the items)	**False positive** (also claimed that migrations were missing; the fields were already in migration 0001)	**Rejected**

Result of the 9 rows: 5 were confirmed real by a failing test (R1, R2, R3a, R3b, R4), 2 were style points that no test can prove (R5, R6), and 2 were false positives (R7, R8). Decisions: 3 accepted and fixed (R1, R2, R3b), 3 deferred (R3a, R4, R5), 3 rejected (R6 low value, R7 and R8 wrong). After the accepted changes:

```
before: Ran 8 tests ... FAILED (failures=5, errors=1)
after : Ran 8 tests ... OK (expected failures=2)      # R3a and R4 are kept as
documented, known failing tests
        and the 35-test Jenkins suite still passes
```

L5.4 [Applied] One function written by hand and one by AI: 10 tests and static analysis

Function: `cart_total(cart, prices)`: the sum of price x quantity for the lines in a cart. **Manual version:** the cart-total loop as it was written in the prototype's `view_cart`, lifted into a function (the project's own code). **AI version:** written from a prompt that described the cart format, legacy values and the limit 20. Both were run against the same 10 tests (`labs/l54/test_total.py`).

Measure	Manual version	AI version
Tests passed (of 10)	**7**	**10**
Failing tests	08 corrupt quantity `'abc'` (crashes with `ValueError`); 09 negative quantity (total -$17.98); 10 quantity capped at 20 ($75.00 instead of $30.00)	none
Statements / branch coverage by the 10 tests	12 / 100 %	10 / 100 %
Cyclomatic complexity (radon)	`cart_total` A (4)	`cart_total` A (3), `_quantity` A (3)
Maintainability index (radon)	A (66.83)	A (66.69)
Security issues (bandit)	none	none
pylint score	7.50 / 10 (unused import `Decimal`, no docstring, long line)	8.00 / 10 (2 long lines)
Returns exact `Decimal`	No: starts from the integer `0`, so an empty cart returns `0` not `Decimal('0.00')`	Yes

> **Evidence note.** SonarQube was not available; the table shows its kind of measures (complexity, maintainability, security, duplication) from radon, bandit, pylint and coverage.

Reading the result. The AI version handled all the edge cases, but only because the prompt listed them; the manual version is exactly what the prototype did and it contains the defect R2 found in L5.3. The 10 tests, not the author, decided the comparison. Static analysis scores were close (7.50 versus 8.00), so they would not have caught the three functional gaps.

L5.5 [Advanced] AI disclosure statement, AI usage policy and licence check

The disclosure statement is in Appendix B. The usage log is in Appendix A.

One-page AI usage policy for the Food Delivery MVP team

#	Rule
1	**Allowed:** drafting stories and test ideas, explaining code, generating boilerplate (Dockerfile, Jenkinsfile), refactoring, reviewing pull requests.

#	Rule
2	**Not allowed:** pasting secrets, passwords, customer names, phones or addresses, or the production database into any AI tool. Use fake data (the seed restaurants and `alice`).
3	**Review:** every AI-written line is read by a human before merge. Code needs a test; a security-relevant change needs a second person.
4	**Verification:** AI facts and numbers are checked against the data or the documentation. An unverified AI statement does not go into a report or a pull request.
5	**Record:** each use is written in the AI usage log (tool, prompt, how verified, decision). The pull request says which parts were AI-assisted.
6	**Accountability:** the developer who merges the code owns it. “The AI wrote it” is not an excuse.
7	**Ethics:** no fake reviews or ratings, no AI image presented as the real dish; sample data is labelled as sample data (Task E).
8	**Licences:** check the licence of any library and any snippet that is not our own; no code copied from an unknown source.
9	**Bias and fairness:** do not use AI to rank customers or restaurants in a way that hides the rule from them; examine suggestions that treat users differently by area, name or language.
10	**Review of this policy:** at every retrospective, and after any incident.

Licence check of libraries and an AI snippet used (read from the installed package metadata):

Item	Licence	Use in the project	Can we use it?
Django 6.1.3	BSD-3-Clause	The web framework (shipped)	Yes; keep the copyright notice
Selenium 4.52	Apache-2.0	UI tests (not shipped)	Yes
psycopg 3 (binary)	LGPL-3.0	PostgreSQL driver in Docker	Yes as an unmodified library; do not modify it without publishing the changes
pylint 4.1	GPL-2.0-or-later	Analysis tool (run on the code, not shipped)	Yes as a tool; its licence does not apply to the code it checks
bandit, coverage, radon	Apache-2.0, Apache-2.0, MIT	Analysis tools (not shipped)	Yes
AI snippet: `add_to_cart` (version 4, L3.1)	No separate licence; it uses only the public Django API	Part of the project code	Used as the project’s own code after review. The AI vendor’s terms (which say who owns the output) must be read by the team; I did not search for similar public code

Note: the repository has no `LICENSE` file; before publishing, the team should choose one (for example MIT) and add it.

Appendices

Appendix A: AI Usage Log

"Claude" = Claude Sonnet 5.5, the main assistant that worked on this report. "Separate session" = a fresh Claude session given only the text shown, so that it could not see my other work. The case names ChatGPT; ChatGPT was not used (see Appendix B).

Step	Tool	Prompt used (summary)	How I verified it	Decision
1	Claude	Read the assignment sheet (6 pages) and the project zip; list every task, question and lab	Re-read the sheet pages a second time before writing each part	Accepted
2	Claude	Inspect the code and run the app; find defects	Ran the app, the Django test client and a browser; screenshots (Figures 10, 13, 14, 15)	Accepted: DEF-01 to DEF-07
3	Claude	Write 10 test cases for cart and order placement (AI-TC-01 to 10)	Ran them against the code; the 2 failures were traced to the lines of code	Accepted
4	Claude	Write the fixed copy: save details and total, coupon, quantity cap, register and login wiring	35 tests and 3 Selenium scenarios pass; migrations check clean	Accepted
5	Claude	Write 12 raw tests for L4.4 and classify them	Judged each against the requirement list; ran them	Kept 8, removed 4
6	Claude, separate session (Haiku)	L1.3 prompt, Tool A	Static comparison with project names, URLs and templates	Used for comparison only
7	Claude, separate session (Sonnet)	L1.3 prompt, Tool B	Same	Used for comparison only
8	Claude, separate session (Haiku) x 4	L3.1 prompt versions 1 to 4 (one fresh session each)	Rubric scoring; version 3 and 4 code run inside a copy of the project with my tests	Version 4 chosen as best
9	Claude, separate session (Sonnet)	L2.4: estimate the 10 stories (no access to my numbers)	Compared with my estimates; checked the biggest gaps against the code	Used as a second opinion
10	Claude, separate session (Sonnet)	Task B: guess the bottleneck from the card dates	Re-computed every number from the data: 4 correct, 1 wrong	Used with the correction
11	Claude, separate session (Sonnet)	L5.3: review the pull-request diff (8 suggestions)	Wrote a test for each suggestion and ran it before and after	Accepted 3, deferred 3, rejected 3
12	Claude	L3.2: refactor `add_to_cart`	7 behaviour tests before and after; a deliberate mutation was caught	Accepted
13	Claude	L1.4: extract requirements from 8 messy notes; classify	Traced every item to a note	5 correct, 4 vague, 3 invented

Step	Tool	Prompt used (summary)	How I verified it	Decision
14	Claude	Task A: 10 interview questions; summary table format	Removed 2 leading questions. The answer table is an illustration to be replaced with real user answers	Questions accepted; answers to be replaced
15	Claude	L5.4: write the AI version of `cart_total`	10 tests, coverage, radon, bandit, pylint	Accepted
16	Claude	Write Jenkinsfile, Dockerfile, docker-compose.yml, `requirements.txt`	`docker compose config` validates; stage commands run locally; `docker compose up` run with PostgreSQL (L3.5), order saved and read back	Accepted; official images not tested
17	Claude	Kanban simulation, dashboard and diagrams (code)	Re-computed lead time by hand (133 / 12 = 11.08); looked at every image	Accepted
18	Claude	Draft the text of the report from the evidence	Every number was checked against the saved logs; statements that were not run are marked “Evidence note”	Accepted; to be read and owned by the student

Appendix B: AI Disclosure Statement

I used **Claude (Anthropic)** as an AI assistant (the case sheet names ChatGPT; I did not use ChatGPT). I used it to read the assignment and my project, to find defects, to write and run tests, to write a corrected copy of the cart and order module, to write the Jenkins, Docker and monitoring files, to draft the report text, and, in separate sessions, to estimate stories, review a pull request, guess a bottleneck and compare outputs. Each use is listed in Appendix A.

I verified the output by running the code and tests (35 Django tests, 3 Selenium scenarios in headless Chromium, a Docker Compose run with PostgreSQL, behaviour tests before and after a refactor), by checking each number in the report against the saved logs, by testing every AI review suggestion before accepting it, and by marking in the report the parts that were not run (SonarQube, Jenkins, official Docker images, real users, classmate sign-off, team poker).

I did not share confidential or personal data with any AI tool. The project uses sample restaurants and a test user only.

I have read this work, I have run the tests myself, and I can explain every answer in an interview.

Signature: _____ Date: _____

Appendix C: Code Listings

C.1 Jenkinsfile

```
pipeline {
    agent any
    triggers { githubPush() }                    // build on every push
    options  { timestamps() }
    stages {
        stage('Checkout') { steps { checkout scm } }
        stage('Build') {
            steps {
                sh 'python3 -m venv .venv'
                sh '. .venv/bin/activate && pip install -r requirements.txt'
                sh '. .venv/bin/activate && python manage.py check && python manage.py
makemigrations --check --dry-run'
            }
        }
        stage('Test') {
            steps {
                sh '. .venv/bin/activate && python manage.py test labs_tests.test_ai10
```

```
labs_tests.test_cart_order labs_tests.test_equivalence
labs_tests.test_ai_review_checks_post -v 2'
            }
        }
        stage('Package') {
            steps {
                sh 'zip -qr foodmvp-${BUILD_NUMBER}.zip config orders restaurants static
manage.py requirements.txt Dockerfile docker-compose.yml'
                archiveArtifacts artifacts: 'foodmvp-*.zip', fingerprint: true
            }
        }
    }
    post {
        failure {
            mail to: 'team@example.com',
                 subject: "FAILED: ${env.JOB_NAME} #${env.BUILD_NUMBER}",
                 body: "Build failed at ${env.BUILD_URL}console - the release was NOT
packaged."
        }
        success { echo 'Build, tests and package passed - artifact archived.' }
    }
}
```

C.2 Dockerfile

```
FROM python:3.13-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]
```

C.3 docker-compose.yml

```
services:
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: foodmvp
      POSTGRES_USER: foodmvp
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:?set POSTGRES_PASSWORD in your shell or .env
(not committed)}
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U foodmvp -d foodmvp"]
      interval: 3s
      timeout: 3s
      retries: 15

  web:
    build: .
    depends_on:
      db:
        condition: service_healthy
    environment:
      POSTGRES_HOST: db
      POSTGRES_DB: foodmvp
      POSTGRES_USER: foodmvp
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      ALLOWED_HOSTS: localhost,127.0.0.1
    ports:
      - "8000:8000"
    command: >
      sh -c "python manage.py migrate --noinput &&
             python manage.py seed_restaurants &&
             python manage.py runserver 0.0.0.0:8000"
```

```
volumes:
  pgdata:
```

C.4 Page objects for the Selenium tests (`pages.py`)

```
"""Page Object Model: one class per page; locators live here and nowhere else."""
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC


class BasePage:
    def __init__(self, driver, base_url):
        self.driver, self.base = driver, base_url


class MenuPage(BasePage):
    ADD_BUTTONS = (By.CSS_SELECTOR, 'a.btn-card')

    def open(self, restaurant_id):
        self.driver.get(f'{self.base}/restaurant/{restaurant_id}/')
        return self

    def add_first_n_items(self, restaurant_id, n):
        for i in range(n):
            self.open(restaurant_id)
            self.driver.find_elements(*self.ADD_BUTTONS)[i].click()
        return self


class CartPage(BasePage):
    COUPON = (By.ID, 'coupon'); APPLY = (By.ID, 'apply-coupon')
    TOTAL = (By.ID, 'total'); CHECKOUT = (By.ID, 'checkout-link')

    def open(self):
        self.driver.get(f'{self.base}/orders/cart/'); return self

    def apply_coupon(self, code):
        self.driver.find_element(*self.COUPON).send_keys(code)
        self.driver.find_element(*self.APPLY).click()
        WebDriverWait(self.driver, 5).until(EC.presence_of_element_located((By.ID,
'subtotal')))  # page reloaded
        return self

    def total(self):
        return self.driver.find_element(*self.TOTAL).text

    def proceed_to_checkout(self):
        self.driver.find_element(*self.CHECKOUT).click(); return
CheckoutPage(self.driver, self.base)


class CheckoutPage(BasePage):
    NAME = (By.NAME, 'full_name'); PHONE = (By.NAME, 'phone_number')
    ADDRESS = (By.NAME, 'address'); SUBMIT = (By.CSS_SELECTOR, 'button.btn-submit')

    def place_order(self, name, phone, address):
        self.driver.find_element(*self.NAME).send_keys(name)
        self.driver.find_element(*self.PHONE).send_keys(phone)
        self.driver.find_element(*self.ADDRESS).send_keys(address)
        self.driver.find_element(*self.SUBMIT).click()
```

C.5 Fixed checkout and cart helpers (`orders/views.py`, excerpt)

```
MAX_QTY = 20  # equivalence class: valid quantity per item is 1..20
COUPONS = {
    'SAVE10': ('percent', Decimal('10')),  # 10 % off
    'FLAT5': ('flat', Decimal('5.00')),    # $5 off
}


def _qty(val):
    """Cart values may be a dict (old format) or an int (current format).
```

```
    Corrupt values count as 0; valid ones are clamped to 0..MAX_QTY (AI review R2)."""
    try:
        raw = val.get('quantity', 0) if isinstance(val, dict) else val
        return max(0, min(int(raw), MAX_QTY))
    except (TypeError, ValueError):
        return 0


def _cart_lines(cart):
    """Return (lines, subtotal, clean_cart) for the session cart; skips bad ids."""
    lines, subtotal, clean = [], Decimal('0.00'), {}
    for item_id, val in cart.items():
        try:
            food = FoodItem.objects.get(pk=int(item_id), is_available=True)
        except (FoodItem.DoesNotExist, ValueError):
            continue
        qty = _qty(val)
        if qty < 1:
            continue
        line_total = food.price * qty
        subtotal += line_total
        lines.append({'food': food, 'quantity': qty, 'subtotal': line_total})
        clean[item_id] = qty
    return lines, subtotal, clean


def discount_for(code, subtotal):
    """Discount amount for a coupon code (never more than the subtotal)."""
    rule = COUPONS.get((code or '').strip().upper())
    if not rule:
        return Decimal('0.00')
    kind, value = rule
    amount = subtotal * value / 100 if kind == 'percent' else value
    return min(amount.quantize(Decimal('0.01')), subtotal)

@login_required
def checkout(request):
    lines, subtotal, clean = _cart_lines(request.session.get('cart', {}))
    if not clean:
        request.session['cart'] = {}
        return redirect('view_cart')
    discount = discount_for(request.session.get('coupon'), subtotal)
    total = subtotal - discount

    if request.method == 'POST':
        name = request.POST.get('full_name', '').strip()
        phone = request.POST.get('phone_number', '').strip()
        address = request.POST.get('address', '').strip()
        if not (name and phone and address):
            return render(request, 'orders/checkout.html',
                          {'total': total, 'total_price': total,
                           'error': 'All delivery fields are required'}, status=400)
        with transaction.atomic():            # AI review R1: all-or-nothing
            order = Order.objects.create(user=request.user, full_name=name,
                                         phone_number=phone, address=address,
                                         total_amount=total)
            for line in lines:
                OrderItem.objects.create(order=order, food_item=line['food'],
                                         quantity=line['quantity'],
price=line['food'].price)
        request.session['cart'] = {}
        request.session.pop('coupon', None)
        return redirect('order_history')
    return render(request, 'orders/checkout.html', {'total': total, 'total_price':
total})
```

Appendix D: Repository Map and How to Reproduce

Path (in `case03/`)	Contents
`original/`	The prototype exactly as submitted (zip contents)
`fixed/`	Corrected copy with coupon feature, review fixes, `Jenkinsfile`, `Dockerfile`, `docker-compose.yml`, `requirements.txt` (the database file is not included; run `migrate` and `seed_restaurants`)
`fixed/labs_tests/`	All tests: `test_ai10`, `test_cart_order`, `test_equivalence`, `test_ai_raw_L44`, `test_ai_review_checks(_post)`, `test_refactor_add_to_cart`, Selenium linear and POM, `pages.py`
`labs/l54/`	Manual and AI `cart_total` and their 10 tests
`labs/kanban/`	Kanban simulation, card CSV and summary
`labs/monitoring/monitor.py`	Monitoring probe (L4.5)
`labs/run_stages.sh`, `labs/dupcheck.py`, `labs/diagrams.py`, `labs/shots_perf.py`, `labs/explore.py`	Pipeline emulation, duplicate detector, figures, screenshots and timing, exploratory session
`evidence/`	Test logs, screenshots, timing and monitor output used in this report

```
cd fixed
python manage.py migrate && python manage.py seed_restaurants
python manage.py test labs_tests.test_ai10 labs_tests.test_cart_order
labs_tests.test_equivalence   # 35 tests
CHROMEDRIVER=<path> python manage.py test labs_tests.test_selenium_pom
# Selenium (needs Chrome)
python labs/kanban/kanban_sim.py
# Kanban table
```